

HASHICORP CERTIFIED · TERRAFORM ASSOCIATE 003

EXAM COMMAND CENTER



Reviews · 261 questions · 3 timed mocks · the shortest path to 70%. Assumes the Master series — this volume reviews and drills, it does not re-teach from zero.

Master Vol 1 ✓
Foundations

Master Vol 2 ✓
Production

Exam Command Center
← this book

 Command Map

Intelligence → arm each domain → drill → simulate → analyze → adjust. Every row jumps; every folio carries you back.

I INTEL + ROUTE

01	Cover	start
02	Map (this page)	here
03	Study route	2 min
04	Exam intelligence	facts+strategy

II DOMAIN REVIEWS (11 pages)

06	D1 IaC concepts · 7%	review
07	D2 Terraform purpose · 8%	review
08	D3 Basics · 9%	review
09	D4 CLI · 8%	review
10	D5 Modules · 8%	review
11	D6 Workflow · 9%	review
12	D7 State · 20% ★	2 pages
14	D8 Configuration · 18% ★	2 pages
16	D9 Cloud/Enterprise · 13%	review

III DOMAIN DRILLS (90 questions)

18	D1–D2 drills · 14 Q	warm cannons
22	D3–D4 drills · 17 Q	basics+CLI
26	D5–D6 drills · 17 Q	modules+flow
30	D7 drills · 18 Q ★	state heavy
35	D8 drills · 16 Q ★	config heavy
39	D9 drills · 8 Q	cloud

IV MOCKS + KEYS + TRACKER

17	Mock briefing + strategy	read first
41	Mock exam 1 · 57 Q	⌚ 60 min
49	Mock exam 2 · 57 Q	⌚ 60 min
57	Mock exam 3 · 57 Q	⌚ 60 min
65	Drill explanations	all 90
95	Mock deep keys	hardest 60
115	Weakness tracker	🏁 end

Intel → Arm → Drill → Simulate → Adjust

From **intel** (know the enemy: 57 questions, 63 seconds each, 40 correct passes) → arm each domain (**D1** through **D9**, heaviest first if time is short: **state 20%** + **config 18%** ≈ 22 questions) → drill (**90 domain questions**) → simulate (**three timed mocks**) → analyze (**keys**) → adjust (**tracker**). The exam rewards recognition; the job rewards understanding — every explanation trains both.

🔑 THE 40-POINT PLAN

Pass = 40/57. The highest-frequency archetypes alone cover it: state purposes + locking (D7), precedence + count/for_each (D8), workflow order (D6), workspaces (D3), modules sources (D5), Sentinel/TFC workspaces (D9). Master those six clusters and the remaining 17 points come from recognition.

☐Route by time available

You have	Do	Skip
One evening	Intel + D7 + D8 reviews + Mock 1	Drills; mocks 2–3
One weekend	All reviews + all drills + Mock 1 + Mock 2	Deep keys except misses
Two weeks	Everything in order; all three mocks timed; tracker green	Nothing

Know the Enemy: 57 Q • 60 Min • 70%

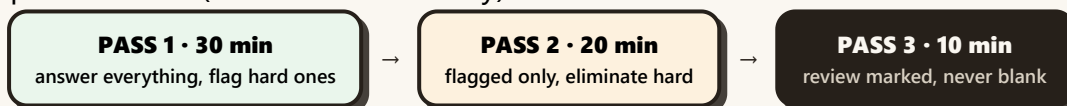
The exam card (memorize)

Fact	Value
Questions	57 • multiple choice + multi-select
Time	60 minutes → ~63 seconds per question
Pass	70% ≈ 40 correct — you may miss 17
Format	Proctored online • ~\$70.50 • valid 2 years

▣ Domain weights = your study budget

Domain	Wt	≈ Qs
D7 State	20%	~11
D8 Configuration	18%	~10
D9 Cloud/Enterprise	13%	~7
D3 Basics • D6 Workflow	9% each	~5+5
D2 Purpose • D4 CLI • D5 Modules	8% each	~5+5+4
D1 IaC concepts	7%	~4

▣ The three-pass method (63-second economy)



Blank = guaranteed zero. A flagged guess beats an empty answer every time.

Morning-Of + Guaranteed-Pass Checklist

The exam tests recognition under time pressure — not depth. Your edge is elimination speed: every distractor below is a documented misconception, and this book has trained each one.

☐ Morning of (the boring list that works)

ID + quiet room + cleared desk ready before login

One mock warm-up (20 questions, untimed) — recognition primed

Arrive 15 min early; test the proctor software first

Water nearby; 60 minutes is a sprint, not a marathon

☐ Guaranteed-pass checklist (40 points, highest frequency first)

#	Archetype	≈ pts	Armed at
1	State purposes + locking + force-unlock	6	D7
2	Precedence ladder + count/for_each + outputs	6	D8
3	Workflow order + plan modes + saved plans	5	D6
4	Workspaces vs directories + TFC workspaces	5	D3 + D9
5	Module sources + pins + registry types	4	D5
6	fmt/validate/console + lock file + TF_LOG	4	D4
7	IaC benefits + TF vs CFN/Ansible	5	D1 + D2
8	Sentinel + VCS runs + cost estimates	5	D9

Rows 1–8 ≈ 40 points = the pass line. Everything else is margin.

Why "as Code" Beats Clicks

IaC manages infrastructure — networks, VMs, load balancers — in a **descriptive model**, versioned like application source. Same model → same environment, every time. The exam tests purpose and benefits, never syntax, here.

✦ THE ESSENCE — what the exam actually asks

Purpose: eliminate manual, unrepeatable provisioning; make environments reproducible

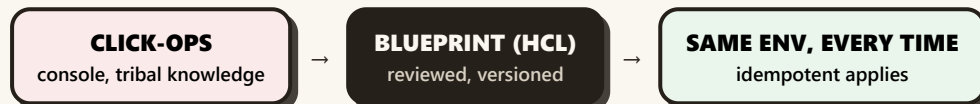
Declarative (declare WHAT — Terraform, CloudFormation) vs **imperative** (script the HOW — Ansible playbooks, bash)

Idempotence: same code applied twice = same result, second run changes nothing

Benefits: version control (git log = audit), repeatability, peer review (PRs), automation (CI), consistency (no snowflakes)

Golden rules: never hand-edit state · never hand-edit `.terraform/` · store code in VCS · treat infra like software

▣ The mental model



The exam loves "which problem does IaC solve?" — answer: manual drift, unrepeatable envs, unaudited change.

⚠ TRAPS

"Imperative = better control" (exam wants declarative for infra) · "IaC means no manual steps ever" (backends/buckets are bootstrapped by hand first) · confusing idempotence with "runs fast".

Drill these → **D1 drills (6 Q)**

What Terraform Is (and Isn't)

A tool for **building, changing, and versioning** infrastructure safely and efficiently — HashiCorp, 2014, Go, single binary, open source (MPL 2.0 line). Manages existing popular providers AND custom in-house solutions.

✦ THE ESSENCE

Three verbs: build · change · version — every exam stem about purpose uses them

Multi-provider universality: AWS + Cloudflare + GitHub in one apply; providers are plugins over RPC

Why HCL declarative: you state end-state; the graph + providers derive ordering and calls

vs CloudFormation: AWS-only, deep integration, no multi-cloud

vs Ansible: imperative config management, no state/plan for infrastructure

vs Pulumi: general-purpose languages instead of HCL — same declarative idea

💡 ELIMINATION KEY

Stems with "single cloud, deep AWS integration" → CloudFormation. "Configure OS/packages on existing hosts" → Ansible. "Version infrastructure across providers" → Terraform.

⚠️ TRAPS

"Terraform is configuration management" (no — provisioning) · "Terraform only manages new infra" (adopts existing via import) ·

 Modern note: 2023+ releases are BUSL; the MPL line continues as OpenTofu.

Drill these → [D2 drills \(8 Q\)](#)

Install, Load, Validate, Isolate

Installing the binary, how configuration loads and merges, workspaces for env isolation, and the two inspection commands. Small domain, highly testable mechanics.

✦ THE ESSENCE

Install: single Go binary (package manager or direct download); upgrade = replace binary, then `init -upgrade`

Load order (last wins): defaults < `TF_VAR_*` < `terraform.tfvars` < `*.auto.tfvars` < `-var-file` < `-var`

Workspaces: `list` / `new` / `select` / `show` / `delete` — one backend, per-env state keys; default is implicit and undeletable

`terraform.workspace` in code = active workspace name (key into per-env maps)

`validate` = syntax + consistency, zero cloud · `fmt` = canonical style

Providers are plugins (RPC); declared in `required_providers`, installed by `init`, pinned by the lock file

▣ The ladder, drawn (last wins, defaults lose)



Read right-to-left when debugging: the rightmost present layer is the value you get.

⚠ TRAPS

"`TF_VAR_*` beats files" (upside-down — files beat env) · "workspaces share state" (isolation is the point) · "validate checks the cloud" (never touches APIs).

Drill these → [D3 drills \(9 Q\)](#)

Everything Outside the Core Loop

Format-on-save hygiene, credentials plumbing, version forensics, the console REPL, and log-driven debugging. The exam loves "which command" and "where is it stored" phrasing here.

✦ THE ESSENCE

fmt (+ `-check` `gate`, `-recursive`) — style machine; **validate** — syntax gate, no cloud

Credentials: `credentials.tfrc.json` (TFC tokens) + env vars (`AWS_*`, `TF_VAR_*`) — never committed secrets

terraform login — writes the TFC/TFE token to the credentials file

Provider versions in use: `.terraform.lock.hcl` (exact builds + hashes)

terraform console — REPL for functions/interpolation (`echo 'format(...)' | terraform console`)

TF_LOG: `OFF`, `ERROR`, `WARN`, `INFO`, `DEBUG`, **TRACE** (most verbose); `TF_LOG_PATH` captures to file

▣ Verbosity ladder + credential map

OFF

→

ERROR

→

WARN

→

INFO

→

DEBUG

→

TRACE

```
$ TF_LOG=TRACE terraform apply 2> debug.log # loudest → file
$ terraform console                        # REPL, no cloud
$ terraform login                          # token → credentials.tfrc.json
```

⚠ TRAPS

"login for AWS" (it's for Terraform Cloud) · "lock file is ignored" (it's committed) · "TRACE is quietest" (it's loudest).

Drill these → [D4 drills \(8 Q\)](#)

Borrowed Code, Pinned Versions

Modules package infrastructure for reuse: the **root** module calls **child** modules by source, feeds them inputs, reads their outputs. Registry (public/private), git, local paths, S3 — sources vary, wiring never does.

✦ THE ESSENCE

Sources: local path · public registry · private registry · git / bitbucket · HTTP URLs · S3

Install: `init` downloads to `.terraform/modules` ; registry modules want `version` pins

Best practices: pin versions, standard structure (`main/variables/outputs`), no hardcoded providers inside

Wiring: root passes inputs → child exposes outputs → root (or stacks) consume `module.NAME.output`

Convention: `outputs.tf` is the module's public interface — reviewers read it first

```
module "network" {  
  source = "terraform-aws-modules/vpc/aws" # registry  
  version = "5.1.0"                        # ← ALWAYS pin registry modules  
  name    = "prod"  
  cidr    = "10.0.0.0/16"  
}  
# use: module.network.vpc_id
```

⚠ TRAPS

Unpinned registry source (floating majors) · provider blocks inside child modules (inherit from root) · reaching past outputs into internals.

Drill these → [D5 drills \(8 Q\)](#)

Write → Init → Plan → Apply

The core loop with its modes and safety rails: plan flavors, saved-plan execution, auto-approve implications, and what interrupting an apply actually costs.

✦ THE ESSENCE

Order: write → init → plan → apply ; plan is read-only, always safe

Modes: default (diff) · -refresh-only (drift preview/sync) · -destroy (teardown preview) · -out=tfplan (saved contract)

apply tfplan executes EXACTLY the reviewed plan; bare **apply** re-plans interactively

-auto-approve: CI behind gates only — hands type confirmation

Stopping apply mid-run: created resources stay, state records them, next apply continues — interruption orphans nothing, but review the partial state

Modules in workflow: init installs them; plans include them; pins freeze them



Dress rehearsal before opening night — reviewed == executed.

⚠ TRAPS

"Saved plan optional in CI" (it's the safety) · "interrupted apply rolls back" (nothing rolls back — state holds partial truth).

Drill these → [D6 drills \(9 Q\)](#)

The Ledger: Purposes, Remote, Locking

One-fifth of your exam. State maps config to reality, carries metadata, speeds plans, and serializes teams. Miss this domain and the math barely passes even if everything else is perfect.

✦ THE ESSENCE — purposes + remote + locking

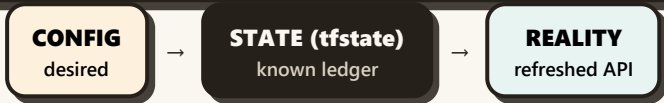
4 purposes: map config→real world · metadata store (IDs/IPs/ARNs) · performance cache · collaboration/locking point

Remote shared truth **sensitive data off** (still plaintext server-side!) · backend options (S3, TFC, Consul...) ·
benefits: · **laptops** locking

Locking: DynamoDB (or native use_lockfile 🗑️) serializes writers — one lock, one writer; stuck locks release by ID after proof of death

State is PLAINTEXT: secrets in state are readable — bucket SSE + tight IAM + Vault/SSM for true secrecy

Never hand-edit the JSON — every fix has a command (list/show/pull/push/rm/mv/import)



Delete the middle box → Terraform forgets everything → duplicates. That one sentence answers five exam questions.

⚠ TRAPS

"State encrypts secrets" (plaintext always) · "locking works on local state" (needs a shared backend) · force-unlock on a live holder (corruption).

Most-tested list → [next page](#) · Drills → [D7 drills \(18 Q\)](#)

Ranked: What the Exam Asks Most

#	Fact	Appears as...
1	State deleted → duplicates on next apply	scenario: "rm tfstate, apply, what happens?"
2	Secrets in state are plaintext	"which is true of sensitive values?"
3	Locking serializes; force-unlock by ID after proof	lock-error output diagnosis
4	rm forgets (lives) vs destroy deletes	"object survives but unmanaged — which cmd?"
5	mv renames tracking, zero rebuild	refactor-without-downtime stems
6	import ADDR ID adopts; config first	click-ops adoption ordering
7	refresh-only previews drift; bare refresh legacy	"show drift, change nothing"
8	Remote = shared + locked + off-laptop	"team of five, one state" scenarios

```
$ terraform state list # what's managed
$ terraform state mv a b # rename tracking $ terraform import ADDR ID
$ terraform force-unlock ID # ghost lock, after proof
```

Drill these → [D7 drills \(18 Q\)](#) · Back to [D7 p1](#)

Address, Depend, Repeat, Vary

Second-heaviest domain: how resources are named and ordered, how copies multiply, how values flow in. Nearly every question here is answered by an address or a symbol.

✦ THE ESSENCE — addressing, deps, repetition, inputs

Addressing: `TYPE.NAME.attr` ; instances `[0]` (count) or `["key"]` (`for_each`); references create graph edges

Dependencies: implicit (references, always preferred) vs explicit (`depends_on` , invisible side-effects only)

count: integer copies, `count.index` ; shrinks kill the tail; 0/1 toggle idiom

for_each: set/map keys, `each.key/value` ; stable identity; never a bare list (wrap `toiset`)

Variables: type + default + description + validation; full ladder (defaults LOSE, `-var` WINS)

Types: string/number/bool · list/set/map · object/tuple/any — sets unordered (`for_each`), lists indexed (`count`)

```
count = 3           → web[0] web[1] web[2]      # position identity
for_each = toiset(..) → web["a"] web["b"]        # key identity
output "x" { value = aws_vpc.main.id }          # publish
locals { env = terraform.workspace }            # derive, don't repeat
```

⚠ TRAPS

`for_each` over a list (must be set/map) · `[0]` on empty (use `one()`) · `depends_on` as default (references first).

Most-tested list → [next page](#) · Drills → [D8 drills \(16 Q\)](#)

Expressions, Splat, Pins

#	Fact	Appears as...
1	Ladder: default < env < tfvars < auto < -var	"which value wins?" with 3+ layers
2	Count shrinks tail; keys hold identity	remove-middle predictions
3	~> = minor-flex, major-freeze	pin operator picks
4	Splat [*] lists all; for reshapes	"all IPs" vs "map of name→IP"
5	Ternary + one() for 0/1 resources	conditional reading
6	Dynamic blocks repeat nested blocks from vars	"N ingress rules from a list"
7	locals derive, resources declare	"where does this expression belong?"
8	Outputs publish; other stacks subscribe	cross-stack value stems

EXAM LENS

D8 rewards address fluency: cover the `TYPE.NAME` and every bracket in the question stem before reading the options — half the distractors die there.

```
{ for k, i in aws_instance.web : k => i.public_ip } # map, not splat
count = var.on ? 1 : 0 + one(x[*].arn)           # 0/1 idiom
version = "~> 5.0" # 5.x yes, 6.0 never    dynamic "ingress" { for_each = var.rules ... }
```

Drill these → [D8 drills \(16 Q\)](#) · Back to [D8 p1](#)

TFC, Sentinel, and the Other Workspaces

Terraform Cloud (SaaS) vs Enterprise (self-hosted): remote runs, VCS-driven plans, governance as code. Headline trap: **TFC workspaces** ≠ **CLI workspaces** — same word, different concept.

✦ THE ESSENCE

TFC = HashiCorp-hosted SaaS · **TFE** = self-hosted (your infra, your compliance boundary)

TFC workspaces bind config + variables + state + run history (NOT the CLI's state-key trick)

VCS integration: PR → speculative plan posted back; merge → apply (gated)

Sentinel policies: advisory (log) · **soft-mandatory** (override with permission) · **mandatory** (hard block)

Registries: public + private module AND provider registries; private = your org's blessed list

Runs: remote operations (plan/apply on TFC agents) · cost estimates on PRs · role-based access · free tier caps

CLI WORKSPACE

state-key label

≠

TFC WORKSPACE

config+vars+state+runs

The exam asks this distinction directly. Same word, different universe.

⚠ TRAPS

"Sentinel advisory blocks" (advisory only logs) · "TFC == TFE" (SaaS vs self-hosted) · agents run *remote* ops, not local ones.

Drill these → [D9 drills \(8 Q\)](#)

How the Mocks Work (Read First)

Three full simulations, 57 questions each, domains scattered like the real exam — **no domain badges on mock questions**, so every stem must be classified by YOU first. That's a trained skill: tag the domain before answering.

Timing discipline

- Write start time. 60 minutes, no pauses.
- Pass 1 (30m): answer all, flag hard.
- Pass 2 (20m): flagged only.
- Pass 3 (10m): review marked. Never blank.

Score bands

- **<40**: back to reviews, not more mocks.
- **40–47**: pass — drill weak domains.
- **48–53**: strong, polish traps.
- **54+**: exam-ready. Book the date.

Answer grids + keys

Each mock ends with a self-marking grid (| Q | your letter |). Grids + hardest-20 deep explanations live in **mock keys**; every domain drill is fully explained at **drill keys**. Explanations teach mechanisms — a WHY that answers 5 future questions, not 1.

FIRST MOVE

Classify, then answer: state wording → D7 · brackets/pins → D8 · "team shares" → D7/D9 · Sentinel/VCS → D9 · "which command" → D4/D6.

D1 IaC + D2 openers

Q-D1-01 • Same config applied twice with no edits. The second plan reports...

- A) No changes — Terraform skips planning
- B) 1 to add, 0 to change, 0 to destroy
- C) 0 to add, 1 to change, 0 to destroy
- D) 0 to add, 0 to change, 0 to destroy

→ Explanation [D1-01](#)

Q-D1-02 • Which is the BEST definition of idempotence?

- A) Code runs faster on repeat
- B) Same code → same infra, re-apply changes nothing
- C) Plans are always empty
- D) State is optional

→ Explanation [D1-02](#)

Q-D1-03 • Your team reviews infra changes in PRs before they happen. Which IaC benefit is this?

- A) Automation
- B) Repeatability
- C) Peer review
- D) Speed

→ Explanation [D1-03](#)

Q-D1-04 • Declarative vs imperative: which pair is correct?

- A) Terraform declares WHAT; scripted playbooks describe HOW
- B) Terraform scripts HOW; playbooks declare WHAT
- C) Both ignore state
- D) Both are imperative

→ Explanation [D1-04](#)

EXAM LENS

D1 answers hide in definitions: idempotence, declarative, version-control benefits.

D1 end • D2 openers

Q-D1-05 • A junior hand-edits production in the console, then runs apply. What does Terraform do?

- A) Detects drift via refresh and proposes to revert it
- B) Ignores console edits forever
- C) Deletes the state file
- D) Locks the console out

→ Explanation **D1-05**

Q-D1-06 • Which file must NEVER be committed to version control?

- A) main.tf
- B) .terraform.lock.hcl
- C) versions.tf
- D) terraform.tfstate

→ Explanation **D1-06**

Q-D2-01 • Terraform is BEST described as...

- A) A tool for building, changing, and versioning infrastructure
- B) A configuration manager for OS packages
- C) An AWS-only deployment service
- D) A container orchestrator

→ Explanation **D2-01**

Q-D2-02 • Your stack spans AWS, Cloudflare, and GitHub in one apply. Which pillar explains why Terraform fits?

- A) Speed and reliability
- B) Platform-agnostic
- C) Coding best practices
- D) Massive community

→ Explanation **D2-02**

EXAM LENS

"Build, change, version" is the purpose answer; second API in the stem means platform-agnostic.

D2 purpose, set 2

Q-D2-03 • A question asks which tool versions infrastructure across clouds. The answer is...

- A) Packer
- B) Ansible
- C) Terraform
- D) Consul

→ Explanation [D2-03](#)

Q-D2-04 • "Configure Nginx and users on 200 existing servers." Best fit?

- A) Terraform
- B) Terraform Cloud
- C) CloudFormation
- D) Ansible

→ Explanation [D2-04](#)

Q-D2-05 • Which statement about the Terraform binary is TRUE?

- A) It is a single Go executable with no runtime dependencies
- B) It requires a Java runtime
- C) It runs only on Linux
- D) It embeds an AWS account

→ Explanation [D2-05](#)

Q-D2-06 • Why is HCL declarative an advantage for infrastructure?

- A) It executes faster than binaries
- B) You state end-state; ordering and API calls are derived
- C) It needs no state
- D) It skips planning

→ Explanation [D2-06](#)

EXAM LENS

Purpose stems answer with the three verbs; tool-fit stems match the layer.

Custom providers • install

Q-D2-07 • A provider for your in-house platform with a REST API. What do you know?

- A) Impossible — only HashiCorp writes providers
- B) Requires forking Terraform
- C) Possible — providers are plugins; anyone can write one
- D) Only via Sentinel

→ Explanation [D2-07](#)

Q-D2-08 • "AWS-only shop, deepest AWS integration wanted." You recommend...

- A) Terraform, for multi-cloud
- B) Packer, for AMIs
- C) Ansible, for agents
- D) CloudFormation, for native depth

→ Explanation [D2-08](#)

Q-D3-01 • Fresh laptop, first day. Which command sequence installs plugins for a new clone?

- A) terraform init
- B) terraform plan
- C) terraform console
- D) terraform fmt

→ Explanation [D3-01](#)

Q-D3-02 • Where do downloaded provider binaries live, and do they belong in git?

- A) .terraform/ — committed
- B) .terraform/ — ignored
- C) terraform.tfstate — ignored
- D) /usr/bin — committed

→ Explanation [D3-02](#)

EXAM LENS

Install = init; binaries = .terraform/ (ignored); lock = committed.

Precedence + workspaces

Q-D3-03 • Values set in defaults, TF_VAR_env, terraform.tfvars, and -var. Which wins?

- A) defaults
- B) TF_VAR_env
- C) -var
- D) terraform.tfvars

→ Explanation **D3-03**

Q-D3-04 • terraform workspace show prints...

- A) the AWS region
- B) all workspaces with resources
- C) the backend bucket
- D) the active workspace name only

→ Explanation **D3-04**

Q-D3-05 • Which workspace CANNOT be deleted?

- A) default
- B) staging
- C) dev
- D) prod

→ Explanation **D3-05**

Q-D3-06 • In code, terraform.workspace gives you...

- A) the backend address
- B) the active workspace name for lookups
- C) a list of resources
- D) the provider version

→ Explanation **D3-06**

EXAM LENS

Ladder ends at -var; default can never be deleted; workspace names key maps.

Pins • fmt gate

Q-D3-07 • terraform validate does what?

- A) Refreshes state
- B) Deploys to a sandbox
- C) Checks syntax + consistency without touching the cloud
- D) Formats files

→ Explanation [D3-07](#)

Q-D3-08 • Two teams need different instance sizes from one config. The primitive is...

- A) a second repository
- B) console edits
- C) hardcoded locals
- D) variables with per-team values

→ Explanation [D3-08](#)

Q-D3-09 • A provider release 6.0.0 must never install; 5.x bug-fixes may. Correct constraint?

- A) version = "~> 5.0"
- B) version = ">= 5.0"
- C) version = "= 5.31.0"
- D) no version key

→ Explanation [D3-09](#)

Q-D4-01 • CI must fail on unformatted HCL without rewriting. Command?

- A) terraform fmt -check
- B) terraform fmt
- C) terraform validate
- D) terraform plan

→ Explanation [D4-01](#)

EXAM LENS

Validate = syntax, no cloud. fmt -check gates; bare fmt rewrites.

Login • lock • console • logs

Q-D4-02 • Where does terraform login store the token?

- A) credentials.tfrc.json
- B) terraform.tfstate
- C) .terraform.lock.hcl
- D) An environment variable

→ Explanation [D4-02](#)

Q-D4-03 • Which file tells you the EXACT provider builds in use?

- A) output.tf
- B) versions.tf
- C) terraform.tfstate
- D) .terraform.lock.hcl

→ Explanation [D4-03](#)

Q-D4-04 • Test a format() expression without applying. Command?

- A) terraform apply -target=null
- B) echo expression | terraform console
- C) terraform refresh
- D) terraform show

→ Explanation [D4-04](#)

Q-D4-05 • Noisiest log level for a provider bug report?

- A) WARN
- B) OFF
- C) INFO
- D) TRACE

→ Explanation [D4-05](#)

EXAM LENS

login writes the credentials file; lock names builds; console tests; TRACE is loudest.

CI habits • first module

Q-D4-06 • Format-on-save in the editor plus what CI gate?

- A) terraform apply
- B) terraform fmt -check
- C) terraform init
- D) terraform destroy

→ Explanation **D4-06**

Q-D4-07 • AWS credentials for Terraform in CI come from...

- A) hardcoded provider blocks
- B) terraform.tfstate
- C) env vars / OIDC role (never committed)
- D) the lock file

→ Explanation **D4-07**

Q-D4-08 • Wrap terraform in Python for a pipeline. What must the wrapper preserve?

- A) The .terraform directory in git
- B) Colored output only
- C) Exit codes and plan-file flow
- D) Interactive prompts

→ Explanation **D4-08**

Q-D5-01 • A registry module must stay on 5.x. Correct source block?

- A) local path with version key
- B) source only
- C) git URL with branch main
- D) source + version = "5.1.0"

→ Explanation **D5-01**

EXAM LENS

Gates fail loudly; secrets never commit; registry modules pin versions.

Sources • install • wiring

Q-D5-02 • Valid module sources include...

- A) only the public registry
- B) local path, registry, git, HTTP/S3 URLs
- C) only local paths
- D) Docker Hub

→ Explanation [D5-02](#)

Q-D5-03 • Where do installed modules land after init?

- A) The registry cache in /tmp
- B) terraform.tfstate
- C) .terraform/modules
- D) Inside main.tf

→ Explanation [D5-03](#)

Q-D5-04 • Root vs child module: which is true?

- A) Outputs flow downward only
- B) Children call the root
- C) Root calls children; children expose outputs the root consumes
- D) There is no root module

→ Explanation [D5-04](#)

Q-D5-05 • A module block is missing version on a registry source. Risk?

- A) Floating majors can break future runs
- B) None — registry pins automatically
- C) Init fails immediately
- D) State is deleted

→ Explanation [D5-05](#)

EXAM LENS

Sources vary, wiring never does: inputs in, outputs out, version pinned.

Registries • the loop

Q-D5-06 • Private registry vs public registry: the difference is...

- A) Language — private uses YAML
- B) Cost — private is always paid
- C) Speed — private skips init
- D) Access scope — private hosts your org blessed modules

→ Explanation **D5-06**

Q-D5-07 • Provider blocks inside child modules: best practice?

- A) Only for AWS
- B) Avoid — inherit from root
- C) Required in every child
- D) Mandatory for pinning

→ Explanation **D5-07**

Q-D5-08 • How does a stack consume `module.vpc.private_subnets[0]`?

- A) As a normal reference — outputs are values like any other
- B) It cannot — outputs are display-only
- C) Only in the console
- D) Only via remote state

→ Explanation **D5-08**

Q-D6-01 • The core workflow in order is...

- A) apply → plan → init → write
- B) init → write → apply → plan
- C) plan → write → init → apply
- D) write → init → plan → apply

→ Explanation **D6-01**

EXAM LENS

Registries differ by scope; the loop never reorders: write, init, plan, apply.

Modes • contracts • interrupts

Q-D6-02 • "Show drift, change nothing." Command?

- A) terraform refresh
- B) terraform plan -refresh-only
- C) terraform apply -auto-approve
- D) terraform destroy

→ Explanation [D6-02](#)

Q-D6-03 • Why apply a saved plan file instead of bare apply?

- A) Faster downloads
- B) Skips init
- C) Executes exactly the reviewed plan, no drift
- D) Avoids state

→ Explanation [D6-03](#)

Q-D6-04 • -auto-approve in production by hand is...

- A) Best practice
- B) Acceptable Fridays
- C) Dangerous — pipelines with gates only
- D) Required for plan

→ Explanation [D6-04](#)

Q-D6-05 • Apply is interrupted halfway (Ctrl+C). What is true?

- A) Created resources stay; state records them; next apply continues
- B) Everything rolls back automatically
- C) State is deleted
- D) The backend is destroyed

→ Explanation [D6-05](#)

EXAM LENS

Refresh-only previews; saved plans bind; interrupts orphan nothing but finish nothing.

Destroy previews • care

Q-D6-06 • terraform plan -destroy -out=kill.plan does what?

- A) Saves a teardown preview for later apply
- B) Destroys immediately
- C) Deletes the state
- D) Removes the backend

→ Explanation [D6-06](#)

Q-D6-07 • A module version bumped upstream. Where does the workflow feel it?

- A) init installs the new code; plan shows the resulting diff
- B) Only in destroy
- C) Nowhere — modules are static
- D) State auto-upgrades

→ Explanation [D6-07](#)

Q-D6-08 • Plan shows -/+ on a database. Before confirming you...

- A) Approve — replacements are routine
- B) Edit state JSON by hand
- C) Delete state and retry
- D) Stop: replacement destroys data; snapshot/migrate first

→ Explanation [D6-08](#)

Q-D6-09 • Which plan mode syncs state toward reality deliberately?

- A) Default plan
- B) apply -refresh-only
- C) plan -destroy
- D) validate

→ Explanation [D6-09](#)

EXAM LENS

-/+ on stateful data stops the line. Destroy previews before destroy applies.

Purposes • remote • secrets • locks

Q-D7-01 • State purposes include all EXCEPT...

- A) Compiling HCL to machine code
- B) Storing metadata like IDs and ARNs
- C) Mapping config to real objects
- D) Speeding up plans via cache

→ Explanation [D7-01](#)

Q-D7-02 • A team of five shares one local state file. First fix?

- A) Email the file weekly
- B) Move to a remote backend with locking
- C) Delete state after each apply
- D) Disable refresh

→ Explanation [D7-02](#)

Q-D7-03 • Sensitive values in state are...

- A) Encrypted by default
- B) Hashed irreversibly
- C) Plaintext — restrict bucket + IAM, use Vault/SSM
- D) Never stored

→ Explanation [D7-03](#)

Q-D7-04 • Second apply starts while the first holds the lock. It...

- A) Proceeds in parallel
- B) Merges both writes
- C) Deletes the lock
- D) Fails with a lock error naming holder and ID

→ Explanation [D7-04](#)

EXAM LENS

D7 is 20%: purposes, plaintext secrets, one-writer locks.

Unlock • rm • mv • import

Q-D7-05 • Lock held by a dead CI runner. Correct recovery?

- A) Re-run apply with -auto-approve until it passes
- B) Delete the DynamoDB row by hand
- C) Destroy and recreate the bucket
- D) Confirm death, then force-unlock by ID, then prove health

→ Explanation [D7-05](#)

Q-D7-06 • terraform state rm aws_instance.old does what?

- A) Stops managing it; the instance keeps running
- B) Destroys the EC2
- C) Renames it
- D) Snapshots it first

→ Explanation [D7-06](#)

Q-D7-07 • Rename web → app with zero rebuild. Command?

- A) terraform state mv aws_instance.web aws_instance.app
- B) Edit state JSON in an editor
- C) terraform destroy -target, then apply
- D) terraform taint and apply

→ Explanation [D7-07](#)

Q-D7-08 • Adopt a hand-built VPC vpc-0ab1. First step?

- A) terraform import immediately
- B) Copy the state file
- C) Delete it and let apply recreate
- D) Write the matching resource block, then import, then plan clean

→ Explanation [D7-08](#)

EXAM LENS

rm abandons, mv renames, import adopts (config first).

Loss • pull/push • drift

Q-D7-09 • State file deleted, then apply. Result?

- A) Clean no-op
- B) Duplicate resources — Terraform forgot everything
- C) Automatic restore from backend
- D) Plan-only mode forever

→ Explanation [D7-09](#)

Q-D7-10 • terraform state pull > backup.json is used to...

- A) Destroy faster
- B) Upgrade providers
- C) Snapshot raw state before risky surgery
- D) Format code

→ Explanation [D7-10](#)

Q-D7-11 • state push is dangerous because it...

- A) Formats JSON
- B) Deletes the backend
- C) Overwrites remote state with your local file
- D) Rotates keys

→ Explanation [D7-11](#)

Q-D7-12 • Drift between config and reality is reconciled by...

- A) Ignoring it — drift self-heals
- B) Deleting .terraform
- C) `fmt -recursive`
- D) refresh during plan, then apply (or explicit refresh-only)

→ Explanation [D7-12](#)

EXAM LENS

Loss duplicates; pull backs up; push overwrites — direction matters.

Forks • hand-edits • outputs • tables

Q-D7-13 • Provider forked (new namespace). Existing state must follow. Command family?

- A) init -upgrade
- B) state replace-provider old/new
- C) fmt -recursive
- D) destroy then apply

→ Explanation [D7-13](#)

Q-D7-14 • Manually editing state JSON to "fix" a value is...

- A) Best practice for speed
- B) Forbidden — every fix has a command; hand edits corrupt ledgers
- C) Required before import
- D) Safe with backups

→ Explanation [D7-14](#)

Q-D7-15 • terraform show -json tfplan | jq .output_changes tells Cl...

- A) Formatting status
- B) Provider download speed
- C) What outputs will change, to gate deploys
- D) The AWS bill

→ Explanation [D7-15](#)

Q-D7-16 • Lock table per env instead of one shared gives you...

- A) Free state encryption
- B) No locking at all
- C) Faster plans
- D) Parallel applies across envs at the cost of more tables to guard

→ Explanation [D7-16](#)

EXAM LENS

Forks need replace-provider; hands off the JSON; outputs gate pipelines.

Ephemeral • keys vs indexes

Q-D7-17 • terraform output -raw db_password in CI logs is a violation because...

- A) Outputs are always encrypted
- B) Sensitive display leaked to logs — mask output, pipe to a store
- C) raw flag deletes state
- D) JSON is required

→ Explanation **D7-17**

Q-D7-18 • ephemeral = true on an output means...

- A) Never stored in state; same-apply read only
- B) Encrypted at rest
- C) Visible to everyone
- D) Stored twice for safety

→ Explanation **D7-18**

Q-D8-01 • aws_subnet.public[0] vs aws_subnet.public["us-east-1a"] differ in...

- A) Index identity vs key identity; only keys survive reordering
- B) Nothing — identical
- C) Speed only
- D) Provider used

→ Explanation **D8-01**

Q-D8-02 • for_each over a bare list fails. Fix?

- A) Use count instead always
- B) Wrap toset(var.list) or build a key map
- C) Delete the list
- D) Add depends_on

→ Explanation **D8-02**

EXAM LENS

Ephemeral never persists; keys outlive positions.

Tails • one() • deps • dynamic

Q-D8-03 • count = 3 → count = 2 destroys...

- A) web[0]
- B) web[1]
- C) web[2] — the tail
- D) All three

→ Explanation [D8-03](#)

Q-D8-04 • Safe reader for a 0/1 counted resource?

- A) x[0] directly
- B) count.index
- C) x[*][0]
- D) one(x[*].attr)

→ Explanation [D8-04](#)

Q-D8-05 • Implicit vs explicit dependencies: prefer...

- A) References (implicit); depends_on only for invisible side-effects
- B) depends_on everywhere
- C) Neither — random order is fine
- D) depends_on instead of references

→ Explanation [D8-05](#)

Q-D8-06 • dynamic "ingress" with for_each over var.rules gives you...

- A) N ingress blocks from one variable, zero HCL edits per rule
- B) Faster plans
- C) No state
- D) Free subnets

→ Explanation [D8-06](#)

EXAM LENS

Tails die first; one() reads 0/1; references order; dynamics repeat.

Splat • required • sets • locals

Q-D8-07 • All public IPs as a list vs name→IP map: correct pair?

- A) Maps are impossible
- B) Both use [0]
- C) [*].public_ip vs { for k,i : k => i.public_ip }
- D) Splat returns a map

→ Explanation [D8-07](#)

Q-D8-08 • Variable with no default and nothing provided: plan...

- A) Deletes state
- B) Prompts (breaks CI) — provide via -var/env/file
- C) Skips the resource
- D) Uses empty string

→ Explanation [D8-08](#)

Q-D8-09 • set(string) vs list(string) for for_each: why sets?

- A) No reason
- B) Faster init
- C) Sets allow duplicates
- D) Stable unique keys; lists re-index on removal

→ Explanation [D8-09](#)

Q-D8-10 • locals { env = terraform.workspace } is an example of...

- A) Hardcoding
- B) Deriving, not repeating — single source per env
- C) A provisioner
- D) Backend config

→ Explanation [D8-10](#)

EXAM LENS

Splat lists, maps key; required vars prompt; sets key stably.

Conditionals • CIDR • merge

Q-D8-11 • Tribunal: ternary count = var.on ? 1 : 0 plus safe read gives...

- A) A syntax error
- B) Always-on resource
- C) Conditional resource with one() output — null when off
- D) Two resources

→ Explanation [D8-11](#)

Q-D8-12 • cidrsubnet("10.0.0.0/16", 8, 10) returns...

- A) An error
- B) 10.0.0.10/32
- C) 10.10.0.0/16
- D) 10.0.10.0/24

→ Explanation [D8-12](#)

Q-D8-13 • 5 subnets needed from a /16: newbits = ?

- A) 3 (8 slots — enough)
- B) 2 (4 slots — too few)
- C) 5
- D) 8

→ Explanation [D8-13](#)

Q-D8-14 • merge(var.tags, { Env = "prod" }) on key collision...

- A) Errors
- B) Right side wins
- C) Left side wins
- D) Both kept

→ Explanation [D8-14](#)

EXAM LENS

0/1 idiom, subnet math, last-wins merge.

Guards • TFC basics

Q-D8-15 • `try(var.obj.attr, "fb")` is for...

- A) Encrypting values
- B) Faster applies
- C) Optional nested access without crashing
- D) Deleting resources

→ Explanation [D8-15](#)

Q-D8-16 • `validation { condition = contains([...], var.env) }` runs...

- A) At plan, rejecting bad input with your message
- B) Never — decorative
- C) Only in the console
- D) After apply

→ Explanation [D8-16](#)

Q-D9-01 • TFC vs TFE: the difference is...

- A) SaaS hosted by HashiCorp vs self-hosted in your boundary
- B) Language used
- C) Only the logo
- D) TFE is older Terraform

→ Explanation [D9-01](#)

Q-D9-02 • A TFC workspace binds...

- A) Only a state key suffix
- B) Config + variables + state + run history
- C) Just the CLI binary
- D) Nothing — display only

→ Explanation [D9-02](#)

EXAM LENS

Guards fail fast; TFC workspaces are full environments, not labels.

VCS runs • Sentinel • registry

Q-D9-03 • PR opened → plan posted back → merge → apply. This is...

- A) State mv
- B) Local-exec
- C) VCS-driven runs
- D) Console mode

→ Explanation [D9-03](#)

Q-D9-04 • Policy blocks deploys missing cost tags, no override. Mode?

- A) Advisory
- B) Soft-mandatory
- C) Disabled
- D) Mandatory

→ Explanation [D9-04](#)

Q-D9-05 • Soft-mandatory vs mandatory Sentinel differs in...

- A) Override possibility — permitted vs never
- B) Speed
- C) Language
- D) Nothing

→ Explanation [D9-05](#)

Q-D9-06 • Private registry gives an org...

- A) Faster CPUs
- B) A blessed, versioned module/provider list
- C) Free state
- D) No init needed

→ Explanation [D9-06](#)

EXAM LENS

VCS drives runs; mandatory blocks; private registries bless.

Costs • agents + checkpoint

Q-D9-07 • Cost estimate on the PR comes from...

- A) Local fmt
- B) State mv
- C) Remote runs analyzing the plan
- D) Console

→ Explanation **D9-07**

Q-D9-08 • Agents in TFC/TFE run...

- A) Local applies on laptops
- B) Only fmt
- C) Nothing — decorative
- D) Remote operations on your infra

→ Explanation **D9-08**

EXAM LENS

Costs ride remote plans; agents bring runs to your network.

DRILL CHECKPOINT

90 drills banked. If D7/D8 misses cluster, re-arm those reviews before mocks — 38% of the exam lives there.

Questions 1–8

🔑 Keys → [M1-01](#) · [M1-02](#) · [M1-03](#) · [M1-04](#) · [M1-05](#) · [M1-06](#) · [M1-07](#) · [M1-08](#)

1. Your editor formats on save, but CI still needs a style gate. Which command fails the build on unformatted HCL?

- A) terraform fmt -check
- B) terraform fmt
- C) terraform validate
- D) terraform plan

2. What does terraform.tfstate primarily allow Terraform to do?

- A) Compile HCL faster
- B) Map configuration to real-world objects
- C) Skip init
- D) Encrypt variables

3. Two applies of unchanged config show 0/0/0. This demonstrates...

- A) declarative syntax
- B) remote backends
- C) idempotence
- D) peer review

4. Where are a managed VPC ID and its CIDR recorded after apply?

- A) Nowhere — re-queried every second
- B) Only in the console output
- C) In versions.tf
- D) In the state file as resource metadata

5. count = 3 is reduced to count = 2. Which instance is destroyed?

- A) web[2]
- B) web[0]
- C) web[1]
- D) All three

6. Which CANNOT be deleted?

- A) A dev workspace
- B) The default workspace
- C) An empty workspace
- D) A staging workspace

7. for_each requires its argument to be...

- A) a list
- B) a number
- C) a set or map
- D) a string

8. Terraform Cloud and Terraform Enterprise differ in...

- A) state format
- B) language: HCL vs YAML
- C) CLI commands entirely
- D) hosting: SaaS vs self-hosted

Questions 9–16

🔑 Keys → [M1-09](#) · [M1-10](#) · [M1-11](#) · [M1-12](#) · [M1-13](#) · [M1-14](#) · [M1-15](#) · [M1-16](#)

9. A policy must block deploys without cost tags, with no override. Sentinel mode?

- A) Mandatory
- B) Advisory
- C) Soft-mandatory
- D) Disabled

10. Token storage after terraform login?

- A) terraform.tfstate
- B) credentials.tfrc.json
- C) .terraform.lock.hcl
- D) An S3 bucket

11. Provider region B while default is region A. The resource needs...

- A) a second Terraform install
- B) depends_on
- C) provider = aws.eu on the block
- D) a second state file

12. TF_VAR_region, terraform.tfvars, and -var all set region. Winner?

- A) defaults
- B) TF_VAR_region
- C) terraform.tfvars
- D) -var

13. State file deleted before apply. Consequence?

- A) Duplicate resources
- B) Safe no-op
- C) Auto-restore
- D) Faster plans

14. "Single cloud, deepest native integration." Best fit?

- A) Terraform
- B) CloudFormation
- C) Ansible
- D) Packer

15. Registry module best practice for versions?

- A) Omit version for latest
- B) Branch main
- C) Pin with version = "x.y.z"
- D) No pin needed

16. PR opened, plan posted back automatically. This is...

- A) Console mode
- B) local-exec
- C) State mv
- D) VCS-driven runs

Questions 17–24

🔑 Keys → [M1-17](#) · [M1-18](#) · [M1-19](#) · [M1-20](#) · [M1-21](#) · [M1-22](#) · [M1-23](#) · [M1-24](#)

17. "Configure users and Nginx on 200 existing VMs." Best fit?

- A) Ansible
- B) CloudFormation
- C) Packer
- D) Terraform

18. "Error acquiring the state lock" appears. First step?

- A) Destroy everything
- B) Delete the DynamoDB row
- C) Read the lock info: who, what op, since when
- D) Disable locking

19. Correct core workflow order?

- A) apply, plan, init, write
- B) write, init, plan, apply
- C) init, apply, plan, write
- D) plan, write, apply, init

20. `aws_subnet.public["us-east-1a"]` addressing implies...

- A) count indexing
- B) A data source
- C) No state
- D) `for_each` keys

21. Same word, different concept in TFC vs CLI?

- A) Workspace
- B) Apply
- C) Plan
- D) State file

22. Safe reader for a possibly-empty counted list?

- A) `x[0]`
- B) `one(x[*].attr)`
- C) `count.index`
- D) `x[*][0]`

23. Saved plan file purpose?

- A) Faster init
- B) Skip state
- C) Execute exactly the reviewed plan
- D) Format code

24. Declarative means...

- A) Script every API call
- B) Manual ordering
- C) No planning needed
- D) State end-state; let the tool derive steps

Questions 25–32

🔑 Keys → [M1-25](#) · [M1-26](#) · [M1-27](#) · [M1-28](#) · [M1-29](#) · [M1-30](#) · [M1-31](#) · [M1-32](#)

25. Remote state benefits include...

- A) Shared truth, locking, secrets off laptops
- B) No locking needed
- C) No backend needed
- D) Free encryption of code

26. Cost estimate on a PR is produced by...

- A) Local fmt
- B) Remote runs analyzing the plan
- C) State mv
- D) The console

27. References between resources create...

- A) Nothing
- B) Costs
- C) Graph edges that order creation
- D) Duplicates

28. terraform workspace show prints...

- A) All resources
- B) The backend type
- C) Plan output
- D) The active workspace name

29. Second run shows 0/0/0 after console tagging. Drift shows as...

- A) A ~ change on the tagged resource
- B) It cannot show
- C) State deletion
- D) A new workspace

30. TFC workspaces bind...

- A) Only a key suffix
- B) Config, variables, state, run history
- C) Nothing
- D) Local binaries

31. version = "~> 5.0" allows...

- A) Only 5.0.0
- B) Any version
- C) 5.x, never 6.0
- D) Only 6.x

32. terraform init does NOT...

- A) Download providers
- B) Configure the backend
- C) Write the lock file
- D) Create real infrastructure

Questions 33–40

🔑 Keys → [M1-33](#) · [M1-34](#) · [M1-35](#) · [M1-36](#) · [M1-37](#) · [M1-38](#) · [M1-39](#) · [M1-40](#)

33. Soft-mandatory Sentinel differs from mandatory in...

- A) Override possibility
- B) Language
- C) Speed
- D) Nothing

34. Which workspace survives every delete attempt?

- A) dev
- B) default
- C) staging
- D) prod

35. Noisiest TF_LOG level?

- A) OFF
- B) INFO
- C) TRACE
- D) WARN

36. Sensitive values in state files are...

- A) Encrypted by default
- B) Hashed
- C) Absent
- D) Plaintext — restrict access

37. Child modules should receive values via...

- A) Input variables from the root call
- B) Console edits
- C) Hardcoded locals
- D) State surgery

38. Pulumi differs from Terraform mainly in...

- A) No state
- B) General-purpose languages vs HCL
- C) AWS-only
- D) No planning

39. Local path, registry, git, HTTP/S3 — these are...

- A) Backend types
- B) Log levels
- C) Module source types
- D) Workspace kinds

40. `{ for k,i : k => i.public_ip }` builds...

- A) A list
- B) An error
- C) A subnet
- D) A name→IP map

Questions 41–48

🔑 Keys → [M1-41](#) · [M1-42](#) · [M1-43](#) · [M1-44](#) · [M1-45](#) · [M1-46](#) · [M1-47](#) · [M1-48](#)

41. terraform validate checks...

- A) Syntax + consistency, no cloud
- B) Real costs
- C) State locks
- D) DNS

42. Golden IaC rule about state files?

- A) Commit them always
- B) Never hand-edit; use state commands
- C) Hand-edit freely
- D) Email them

43. TF_VAR_tags for a map variable needs...

- A) Bare braces
- B) Double quotes only
- C) Single-quoted HCL
- D) A tfvars file only

44. terraform state pull is for...

- A) Faster applies
- B) Formatting
- C) Provider upgrade
- D) Raw JSON snapshot before surgery

45. Private registry gives...

- A) Blessed org modules/providers
- B) Faster CPUs
- C) Free state
- D) No init

46. terraform show -json tfplan | jq .output_changes gates...

- A) Formatting
- B) Deploys on output changes
- C) Billing
- D) DNS

47. TFC agents run...

- A) Laptop applies
- B) Nothing
- C) Remote operations on your infra
- D) Only fmt

48. merge(a, b) on colliding keys...

- A) Errors
- B) Left wins
- C) Drops both
- D) Right wins

Questions 49–56

🔑 Keys → [M1-49](#) · [M1-50](#) · [M1-51](#) · [M1-52](#) · [M1-53](#) · [M1-54](#) · [M1-55](#) · [M1-56](#)

49. Interrupted apply (Ctrl+C) leaves...

- A) Partial state; next apply continues
- B) Full rollback
- C) Deleted backend
- D) No trace

50. validation blocks run...

- A) After apply
- B) At plan, rejecting bad input
- C) In the console only
- D) Never

51. -auto-approve belongs...

- A) On laptops in prod
- B) Everywhere
- C) In gated pipelines only
- D) Nowhere

52. terraform workspace new prod fails: exists. Meaning?

- A) State corrupted
- B) Init needed
- C) Backend down
- D) Name taken — select it instead

53. VCS integration posts plans...

- A) Back on the PR
- B) To email
- C) To state
- D) Nowhere

54. depends_on is for...

- A) Every reference
- B) Invisible side-effects only; references first
- C) Deleting state
- D) Formatting

55. terraform console is...

- A) A backend
- B) A REPL for expressions, no cloud
- C) A deployment tool
- D) A registry

56. force-unlock needs...

- A) Nothing
- B) Fmt first
- C) A new bucket
- D) The lock ID after proof the holder is dead

Question 57 + answer sheet

57. Free-tier TFC caps push you to...
- A) Paid tier or scoped usage
 - B) Share tokens
 - C) Commit state
 - D) Ignore limits

→ Key **M1-57**

☐ Mock 1 answer sheet (mark, then check at **keys**)

Q	Yours	Q	Yours	Q	Yours
1	☐	2	☐	3	☐
4	☐	5	☐	6	☐
7	☐	8	☐	9	☐
10	☐	11	☐	12	☐
13	☐	14	☐	15	☐
16	☐	17	☐	18	☐
19	☐	20	☐	21	☐
22	☐	23	☐	24	☐
25	☐	26	☐	27	☐
28	☐	29	☐	30	☐
31	☐	32	☐	33	☐
34	☐	35	☐	36	☐
37	☐	38	☐	39	☐
40	☐	41	☐	42	☐
43	☐	44	☐	45	☐
46	☐	47	☐	48	☐
49	☐	50	☐	51	☐
52	☐	53	☐	54	☐
55	☐	56	☐	57	☐

Questions 1–8

🔑 Keys → [M2-01](#) · [M2-02](#) · [M2-03](#) · [M2-04](#) · [M2-05](#) · [M2-06](#) · [M2-07](#) · [M2-08](#)

1. State purposes do NOT include...

- A) Compiling providers
- B) Metadata storage
- C) Mapping config to reality
- D) Plan performance

2. Debug a provider with maximum logs to a file.

- A) TF_LOG=OFF
- B) TF_LOG=TRACE ... 2> debug.log
- C) terraform fmt
- D) rm .terraform

3. "Same environment from the same model, every time" is...

- A) Snowflake servers
- B) Manual clicking
- C) Idempotence via IaC
- D) Untracked change

4. TFC free tier limits push heavy teams to...

- A) Local backends only
- B) Committing state
- C) Disabling plans
- D) Paid tier or scoped usage

5. `aws_instance.web["blue"]` implies...

- A) `for_each` keys
- B) count
- C) No backend
- D) A provisioner

6. Pulumi vs Terraform: core difference?

- A) General-purpose languages vs HCL
- B) No state in Pulumi
- C) Pulumi is AWS-only
- D) Terraform has no plan

7. Registry child module needs a bug-fix version safely. Pin?

- A) Branch latest
- B) No pin
- C) `version = "~> 2.0"`
- D) `version = ">= 1.0"`

8. `TF_VAR_count="three"` for a number var: result?

- A) Auto-corrects to 3
- B) Destroys state
- C) Ignored silently
- D) Type error at plan

Questions 9–16

🔑 Keys → [M2-09](#) · [M2-10](#) · [M2-11](#) · [M2-12](#) · [M2-13](#) · [M2-14](#) · [M2-15](#) · [M2-16](#)

9. Write → init → plan → apply. Skipping plan risks...

- A) Faster runs only
- B) Unreviewed changes executing blind
- C) Nothing
- D) Better state

10. Golden rule: infra change without a PR is...

- A) Unreviewed — violates IaC peer review
- B) Faster
- C) Required
- D) Encrypted

11. DynamoDB lock table deleted. Applies now...

- A) Run faster
- B) Encrypt state
- C) Race without serialization — restore it
- D) Skip init

12. terraform output -json | jq .vpc_id.value is for...

- A) Formatting
- B) Init
- C) Locking
- D) CI/script consumption of exports

13. Speculative plan on a PR comes from...

- A) VCS-driven runs
- B) local-exec
- C) fmt
- D) console

14. Lock error names ci-runner-9, job red for an hour. Move?

- A) Wait forever
- B) Confirm death, force-unlock ID, prove health
- C) Delete bucket
- D) Reboot laptop

15. Public registry vs private module registry: use private for...

- A) Avoiding init
- B) Faster downloads only
- C) Blessed org versions
- D) Public code only

16. values(aws_subnet.public)[*].id returns...

- A) One ID
- B) A map
- C) Nothing
- D) All IDs as a list

Questions 17–24

🔑 Keys → [M2-17](#) · [M2-18](#) · [M2-19](#) · [M2-20](#) · [M2-21](#) · [M2-22](#) · [M2-23](#) · [M2-24](#)

17. terraform workspace list shows...

- A) All workspaces, * marking active
- B) Only the current workspace
- C) Resource counts
- D) AWS costs

18. Advisory Sentinel policy violation...

- A) Blocks
- B) Logs only, run continues
- C) Deletes state
- D) Bills extra

19. lookup(var.amis, var.region, "ami-fallback") does...

- A) Crashes on miss
- B) Deletes AMIs
- C) Returns fallback on miss
- D) Ignores region

20. Manual console SG edit appears in next plan as...

- A) Nothing
- B) An error, always
- C) A new workspace
- D) A ~ change to reconcile

21. terraform apply tfplan guarantees...

- A) Reviewed == executed
- B) No state
- C) No backend
- D) Faster init

22. terraform state show is used to...

- A) Delete resources
- B) Display one resource attributes
- C) Format code
- D) Create workspaces

23. Mandatory policy with missing tags...

- A) Warns only
- B) Applies anyway
- C) Hard-blocks the run
- D) Deletes the policy

24. -refresh-only mode is for...

- A) Destroying
- B) Init
- C) Formatting
- D) Previewing/syncing drift explicitly

Questions 25–32

🔑 Keys → [M2-25](#) · [M2-26](#) · [M2-27](#) · [M2-28](#) · [M2-29](#) · [M2-30](#) · [M2-31](#) · [M2-32](#)

25. apply -refresh-only writes...

- A) Synced state toward reality
- B) Nothing
- C) A new backend
- D) Plan to trash

26. Remote operations (TFC agents) mean...

- A) No history kept
- B) Plans/applies run off-laptop with history
- C) Local-only runs
- D) No variables

27. New workspace starts with...

- A) Copied prod state
- B) All resources
- C) Empty state — apply to fill
- D) An error

28. coalesce(var.a, var.b, "fb") returns...

- A) Always "fb"
- B) Nothing
- C) A list
- D) First non-null/non-empty

29. terraform fmt -recursive covers...

- A) Subfolders too
- B) One file
- C) State
- D) The registry

30. Module outputs flow...

- A) Downward only
- B) Upward to root/consumers
- C) Nowhere
- D) To S3 directly

31. Root module is...

- A) The state file
- B) A registry only
- C) The entrypoint config that calls children
- D) The lock file

32. -destroy plan mode previews...

- A) Creation
- B) Init
- C) Formatting
- D) Teardown for later apply

Questions 33–40

🔑 Keys → [M2-33](#) · [M2-34](#) · [M2-35](#) · [M2-36](#) · [M2-37](#) · [M2-38](#) · [M2-39](#) · [M2-40](#)

33. *.auto.tfvars vs terraform.tfvars differ in...

- A) Layered lexical overlays vs single file
- B) Nothing
- C) Only comments
- D) Speed

34. echo expression | terraform console tests...

- A) Full applies
- B) Functions/interpolation with no cloud
- C) State locks
- D) Billing

35. Ansible manages infra state like Terraform? No — because...

- A) It is newer
- B) It is faster
- C) It has no state/plan for infra
- D) It costs more

36. Dynamic blocks repeat...

- A) Providers
- B) Workspaces
- C) Backends
- D) Nested blocks from variables

37. TFC cost estimates appear...

- A) On PRs from remote plans
- B) In state
- C) In fmt output
- D) Never

38. Bare apply in prod is replaced by...

- A) destroy
- B) apply tfplan after review
- C) rm state
- D) console edits

39. Soft-mandatory violation with override rights...

- A) Blocks always
- B) Deletes policy
- C) Proceeds with permission
- D) Ignores code

40. terraform state list shows...

- A) Nothing useful
- B) AWS bills
- C) Provider source
- D) Every managed address

Questions 41–48

🔑 Keys → [M2-41](#) · [M2-42](#) · [M2-43](#) · [M2-44](#) · [M2-45](#) · [M2-46](#) · [M2-47](#) · [M2-48](#)

41. Module pin missing on registry source risks...

- A) Floating majors breaking runs
- B) Nothing
- C) Instant failure
- D) State loss

42. terraform.tfvars committed with secrets. Fix?

- A) Nothing — normal
- B) Gitignore + example template + secret store
- C) Delete git
- D) Encrypt filenames

43. depends_on belongs where?

- A) Everywhere
- B) Nowhere
- C) Invisible side-effects only
- D) In outputs

44. api.example.com CNAME via Cloudflare provider in one apply shows...

- A) Lock failure
- B) A bug
- C) Drift
- D) Multi-provider power

45. terraform validate needs...

- A) Cloud credentials
- B) No cloud — syntax only
- C) A backend
- D) State

46. Backend block with var.* fails because...

- A) Vars are broken
- B) Backends take literals/-backend-config only
- C) S3 is down
- D) Init is slow

47. for_each = toset(var.list) converts...

- A) Map to string
- B) Number to bool
- C) List to stable set
- D) Nothing

48. terraform show reads...

- A) DNS
- B) The future
- C) Billing
- D) State or plan files human/JSON

Questions 49–56

🔑 Keys → [M2-49](#) · [M2-50](#) · [M2-51](#) · [M2-52](#) · [M2-53](#) · [M2-54](#) · [M2-55](#) · [M2-56](#)

49. TRACE vs INFO: TRACE is...

- A) Loudest, full RPC detail
- B) Off
- C) Quieter
- D) Errors only

50. Orphaned EC2 found running, untracked. First command?

- A) destroy
- B) terraform state list to confirm absence, then import flow
- C) rm -rf
- D) fmt

51. TFC run history lives...

- A) Nowhere
- B) Locally only
- C) In the workspace (config+vars+state+runs)
- D) In git

52. Module output consumed as...

- A) Via email
- B) Only via console
- C) Never
- D) module.NAME.output like any value

53. "Same model, same env" is IaC...

- A) Risk
- B) Slowness
- C) Cost
- D) Repeatability

54. Workspaces isolate...

- A) Nothing
- B) State per env under one backend
- C) Providers
- D) Binaries

55. -out=tfplan then apply tfplan gives...

- A) Faster init
- B) No state
- C) Reviewed == executed
- D) No backend

56. one(null_resource.none[*].id) returns...

- A) An error
- B) A list
- C) Zero
- D) null

Question 57 + answer sheet

57. Old tutorial says "terraform refresh". Modern equivalent?

- A) terraform plan -refresh-only
- B) Nothing — removed
- C) terraform destroy
- D) terraform fmt

→ Key **M2-57**

☐ Mock 2 answer sheet (mark, then check at **keys**)

Q	Yours	Q	Yours	Q	Yours
1	☐	2	☐	3	☐
4	☐	5	☐	6	☐
7	☐	8	☐	9	☐
10	☐	11	☐	12	☐
13	☐	14	☐	15	☐
16	☐	17	☐	18	☐
19	☐	20	☐	21	☐
22	☐	23	☐	24	☐
25	☐	26	☐	27	☐
28	☐	29	☐	30	☐
31	☐	32	☐	33	☐
34	☐	35	☐	36	☐
37	☐	38	☐	39	☐
40	☐	41	☐	42	☐
43	☐	44	☐	45	☐
46	☐	47	☐	48	☐
49	☐	50	☐	51	☐
52	☐	53	☐	54	☐
55	☐	56	☐	57	☐

Questions 1–8

🔑 Keys → [M3-01](#) · [M3-02](#) · [M3-03](#) · [M3-04](#) · [M3-05](#) · [M3-06](#) · [M3-07](#) · [M3-08](#)

1. "Build, change, version infrastructure" describes...

- A) Terraform
- B) Packer
- C) Ansible
- D) Git

2. Private provider registry is for...

- A) Public mirrors only
- B) Blessed org providers
- C) Skipping init
- D) Billing

3. terraform init -upgrade does what?

- A) Destroys state
- B) Formats code
- C) Re-resolves newest allowed providers, rewrites lock
- D) Creates workspaces

4. Saved destroy plan via...

- A) login
- B) fmt
- C) console
- D) plan -destroy -out=kill.plan

5. Module source pinned with version is...

- A) Best practice for registry modules
- B) Optional
- C) Forbidden
- D) Slow

6. Remote state benefits exclude...

- A) Shared truth
- B) Local-only secrecy
- C) Locking
- D) Off-laptop secrets

7. toset(var.list) is needed because for_each takes...

- A) Lists happily
- B) Numbers
- C) Sets or maps only
- D) Nothing

8. Ghost lock recovery starts with...

- A) Reboot
- B) Deleting the table
- C) Destroy
- D) Reading lock info

Questions 9–16

🔑 Keys → [M3-09](#) · [M3-10](#) · [M3-11](#) · [M3-12](#) · [M3-13](#) · [M3-14](#) · [M3-15](#) · [M3-16](#)

9. Provider credentials belong in...

- A) Env vars / SSO, never git
- B) Committed blocks
- C) State outputs
- D) Lock file

10. "OS users and packages on live hosts" →

- A) Terraform
- B) Ansible
- C) CloudFormation
- D) TFC

11. Conditional resource idiom?

- A) No conditionals exist
- B) count = always
- C) count = var.on ? 1 : 0
- D) Destroy first

12. Peer review of infra happens via...

- A) Console clicks
- B) State edits
- C) Phone calls
- D) VCS PRs on HCL

13. values(map)[*].id collects...

- A) All values IDs as a list
- B) Keys
- C) Nothing
- D) One string

14. Agents run plans...

- A) On laptops
- B) Remotely with history
- C) Never
- D) In fmt

15. Explicit credentials in a committed block risk...

- A) Nothing
- B) Faster plans
- C) Scraped secrets in minutes
- D) Better state

16. Stale local state vs changed reality reconciles via...

- A) New workspace
- B) Deleting config
- C) Fmt
- D) Refresh during plan

Questions 17–24

🔑 Keys → [M3-17](#) · [M3-18](#) · [M3-19](#) · [M3-20](#) · [M3-21](#) · [M3-22](#) · [M3-23](#) · [M3-24](#)

17. Speculative PR plans come from...

- A) VCS-driven runs
- B) Cron fmt
- C) Local state
- D) Email hooks

18. terraform state show vpc reads...

- A) All bills
- B) One resource attributes
- C) Provider source
- D) Nothing

19. merge() collision winner is...

- A) Left
- B) Error
- C) Right
- D) Neither

20. terraform.tfvars vs *.auto.tfvars differ in...

- A) Nothing
- B) Color
- C) Speed
- D) Single file vs layered lexical overlays

21. Root passes inputs to children via...

- A) Module call arguments
- B) State surgery
- C) Console
- D) Email

22. Console drift on tags appears as...

- A) No output
- B) ~ change
- C) New backend
- D) Crash

23. coalesce picks...

- A) Last null
- B) All values
- C) First non-null
- D) None

24. *.tfvars.example committed while real tfvars ignored because...

- A) Speed
- B) State needs it
- C) Required by law
- D) Template shared, secrets stay local

Questions 25–32

🔑 Keys → [M3-25](#) · [M3-26](#) · [M3-27](#) · [M3-28](#) · [M3-29](#) · [M3-30](#) · [M3-31](#) · [M3-32](#)

25. Interrupted mid-apply: next step?

- A) Review partial state, re-apply to continue
- B) Delete everything
- C) New backend
- D) Fmt

26. "No manual snowflakes" is IaC...

- A) Cost
- B) Consistency
- C) Slowness
- D) Risk

27. Go binary, no runtime describes...

- A) The provider only
- B) State files
- C) The Terraform CLI distribution
- D) Sentinel

28. Same config, two applies, both green 0/0/0. Property?

- A) Import
- B) Drift
- C) Taint
- D) Idempotence

29. try() guards...

- A) Optional access without crashing
- B) Billing
- C) Locks
- D) DNS

30. Graph edges derive from...

- A) File order
- B) References
- C) Comments
- D) Filenames

31. console tests...

- A) Locks
- B) Full deploys
- C) Expressions with no cloud
- D) Bills

32. TFC workspace binds...

- A) Binaries
- B) A key suffix only
- C) Nothing
- D) Config, vars, state, runs

Questions 33–40

🔑 Keys → [M3-33](#) · [M3-34](#) · [M3-35](#) · [M3-36](#) · [M3-37](#) · [M3-38](#) · [M3-39](#) · [M3-40](#)

33. Multi-cloud in one apply is possible because...

- A) Providers are plugins over RPC
- B) One giant binary
- C) No state
- D) YAML

34. Cost control on PRs via...

- A) Fmt
- B) Remote-run estimates
- C) State rm
- D) Taint

35. Splat [*] over instances gives...

- A) A map
- B) An error
- C) A list of the attribute
- D) A backend

36. Ghost lock: first read...

- A) Fmt
- B) Nothing
- C) Billing
- D) Lock info: who/what/since

37. Log verbosity for deep debugging?

- A) TRACE
- B) ERROR
- C) OFF
- D) WARN

38. apply -refresh-only syncs...

- A) Code style
- B) State toward reality
- C) Providers
- D) Regions

39. Advisory policy breach...

- A) Blocks
- B) Deletes
- C) Logs, run continues
- D) Bills

40. Partial apply then re-apply...

- A) Rolls back
- B) Forks code
- C) Wipes backend
- D) Continues from recorded state

Questions 41–48

🔑 Keys → [M3-41](#) · [M3-42](#) · [M3-43](#) · [M3-44](#) · [M3-45](#) · [M3-46](#) · [M3-47](#) · [M3-48](#)

41. "Golden image with Nginx baked" is built by...

- A) Packer
- B) Terraform provisioners
- C) Console
- D) State mv

42. for_each over ["a","b"] fails: fix?

- A) Delete list
- B) toset() wrap
- C) depends_on
- D) Count only

43. TF_VAR for objects needs...

- A) Bare words
- B) Double binaries
- C) Single-quoted HCL
- D) No quoting

44. Child output consumed as...

- A) Never
- B) Console only
- C) Email
- D) module.NAME.output

45. terraform.tfvars purpose?

- A) Auto-loaded variable values
- B) Backend only
- C) Locking
- D) Billing

46. Lock table per env buys...

- A) Nothing
- B) Parallel applies, more tables
- C) No init
- D) Free state

47. validation blocks run...

- A) Never
- B) After destroy
- C) At plan
- D) In git

48. -destroy plan mode is for...

- A) Login
- B) Creation
- C) Fmt
- D) Teardown preview

Questions 49–56

🔑 Keys → [M3-49](#) · [M3-50](#) · [M3-51](#) · [M3-52](#) · [M3-53](#) · [M3-54](#) · [M3-55](#) · [M3-56](#)

49. New workspace state starts...

- A) Empty — apply fills it
- B) With prod data
- C) With errors
- D) Locked forever

50. TFC workspace binds...

- A) Key suffix only
- B) Config, vars, state, runs
- C) Nothing
- D) Binaries

51. Login writes token to...

- A) State
- B) Lock file
- C) credentials.tfrc.json
- D) tfvars

52. locals derive values to...

- A) Duplicate code
- B) Slow plans
- C) Skip state
- D) Avoid repetition, single source

53. terraform output shows...

- A) Exports after apply
- B) Billing
- C) Locks
- D) Nothing

54. Pin registry modules because...

- A) Speed
- B) Floating majors break runs
- C) State needs it
- D) Fmt does

55. Soft-mandatory with rights...

- A) Blocks
- B) Deletes
- C) Proceeds permitted
- D) Ignores

56. -auto-approve belongs...

- A) Laptops in prod
- B) Never anywhere
- C) Always
- D) Gated pipelines only

Question 57 + answer sheet

57. S3 backend state with secrets is...
- A) Plaintext — restrict + Vault/SSM
 - B) Encrypted by Terraform
 - C) Absent
 - D) Hashed

→ Key **M3-57**

▣ Mock 3 answer sheet (mark, then check at **keys**)

Q	Yours	Q	Yours	Q	Yours
1	☐	2	☐	3	☐
4	☐	5	☐	6	☐
7	☐	8	☐	9	☐
10	☐	11	☐	12	☐
13	☐	14	☐	15	☐
16	☐	17	☐	18	☐
19	☐	20	☐	21	☐
22	☐	23	☐	24	☐
25	☐	26	☐	27	☐
28	☐	29	☐	30	☐
31	☐	32	☐	33	☐
34	☐	35	☐	36	☐
37	☐	38	☐	39	☐
40	☐	41	☐	42	☐
43	☐	44	☐	45	☐
46	☐	47	☐	48	☐
49	☐	50	☐	51	☐
52	☐	53	☐	54	☐
55	☐	56	☐	57	☐

✓ D1-01 • D1-02 • D1-03

Q-D1-01 ✓ **D** — 0 to add, 0 to change, 0 to destroy — plus the "No changes" line.

🧠 **WHY** — Refresh re-read reality, compare found zero drift: desired == known == real. An empty diff is the healthy steady state, not a skipped run. Idempotence means the second apply is a no-op by design.

✗ **A/B/C** — any nonzero count claims changes that do not exist

✗ **D** — "skips planning" mistakes the outcome for the process — Terraform planned fully, then did nothing

🎓 **EXAM** — Same code twice → 0/0/0. Any nonzero number needs a cause (edits, drift, taint).

Q-D1-02 ✓ **B** — Idempotence = same code → same infra, re-apply changes nothing.

🧠 **WHY** — It is a property of the RESULT across runs, not of speed, plans, or state files. "Faster on repeat" describes caching; "plans always empty" ignores first applies and real changes.

✗ **A** — speed is irrelevant — a slow idempotent apply is still idempotent

✗ **C** — first applies and drifted states produce full diffs

✗ **D** — state is the mechanism that MAKES idempotence possible

🎓 **EXAM** — When a stem says "twice, unchanged", answer idempotence before finishing the sentence.

Q-D1-03 ✓ **C** — Reviewing infra in PRs before it happens = peer review.

🧠 **WHY** — IaC puts infrastructure into reviewable text, so the same PR machinery that guards app code guards prod. Automation runs it; repeatability stabilizes it; neither involves human judgment.

✗ **A** — automation executes — it does not judge

✗ **B** — repeatability is about identical outcomes, not approval

✗ **D** — speed is never the point of a review gate

🎓 **EXAM** — PR + infra + "before" → peer review. "Same every time" → repeatability.

✓ D1-04 • D1-05 • D1-06

Q-D1-04 ✓ **A** — Declarative states WHAT (Terraform); imperative scripts HOW (playbooks).

🧠 WHY — Terraform describes end-state and derives ordering plus API calls; Ansible-style playbooks list steps to execute. The exam pairs the paradigm with the tool without exception.

✗ B — exact inversion — the exam's favorite trap shape

✗ C — playbooks are the imperative half of the pair

✗ D — state is central to Terraform; playbooks simply lack it

🎓 EXAM — WHAT=HCL/Terraform, HOW=steps/playbooks. Inverted options are always wrong.

Q-D1-05 ✓ **A** — Console tag appears as a ~ change; refresh caught the drift.

🧠 WHY — Refresh imports REAL into the diff, so hand edits surface as update-in-place lines against code. The plan is the drift report nobody asked anyone to write.

✗ B — nothing is invisible to refresh — that is its entire job

✗ C — state is read, never deleted, by planning

✗ D — no console can lock Terraform out of reading APIs

🎓 EXAM — Hand edit + next plan = ~ line. Resolve by code (apply) or console (revert).

Q-D1-06 ✓ **D** — Never commit terraform.tfstate.

🧠 WHY — State holds secrets in plaintext plus the keys to everything. versions.tf and the lock file SHOULD commit (reproducibility); main.tf is the whole point of version control.

✗ A — main.tf is exactly what VCS is for

✗ C — versions.tf pins belong in git

✗ D — lock file guarantees identical providers

🎓 EXAM — State + secrets + git = breach. tfstate, tfvars values, and keys stay local.

✓ D2-01 • D2-02 • D2-03

Q-D2-01 ✓ **A** — Build, change, version — the three verbs.

💡 WHY — Terraform exists to take infrastructure through its whole lifecycle under version control. "Configuration manager" describes Ansible; "AWS-only" describes CloudFormation; "orchestrator" describes Nomad/K8s.

✗ B — OS packages = config management, wrong layer

✗ C — AWS-only contradicts multi-provider universality

✗ D — containers are someone else's job

🎓 EXAM — Purpose stems answer with the three verbs, every time.

Q-D2-02 ✓ **B** — Platform-agnostic wins the moment a second API enters.

💡 WHY — AWS + Cloudflare + GitHub in one apply is impossible for CloudFormation and pointless to split across tools. Community, speed, and practices are all true of Terraform and all miss the question asked.

✗ A — speed never answers a scope question

✗ C — practices govern HOW, not WHERE

✗ D — community is real and irrelevant here

🎓 EXAM — Second API in the stem → platform-agnostic. Always.

Q-D2-03 ✓ **C** — Versions-infrastructure-across-clouds = Terraform.

💡 WHY — Packer bakes images, Ansible configures hosts, Consul discovers services. Only one tool on the list versions heterogeneous infrastructure.

✗ A — Packer builds AMIs; it manages no live infra

✗ B — Ansible has no state/plan for infrastructure

✗ D — Consul is service discovery, not provisioning

🎓 EXAM — Tool-fit questions eliminate by layer: provision, configure, bake, discover.

✓ D2-04 • D2-05 • D2-06

Q-D2-04 ✓ **D** — 200 existing servers needing users and Nginx = Ansible.

💡 WHY — The work is OS-level configuration of long-lived hosts — the config-management layer, where idempotent modules beat provisioning tools. Terraform provisions; it does not babysit fleets.

✗ A — Terraform would need a provider per package manager — absurd

✗ B — CloudFormation is AWS-only AND provisioning-layer

✗ C — TFC runs Terraform; wrong tool, fancier venue

🎓 EXAM — "Existing hosts + packages/users" → Ansible. "New infra + lifecycle" → Terraform.

Q-D2-05 ✓ **A** — Single Go executable, zero runtime dependencies.

💡 WHY — One static binary is the whole distribution story: download, run, no interpreter, no agents. Java runtimes, Linux-only limits, and embedded AWS accounts are all fictions.

✗ B — Go compiles statically — no JVM anywhere

✗ C — all major OSes ship builds

✗ D — it calls AWS APIs; it contains no account

🎓 EXAM — Distribution questions test "single binary, Go, no runtime" — one fused fact.

Q-D2-06 ✓ **B** — Declare end-state; ordering and API calls derive.

💡 WHY — HCL says WHAT the world should look like; the graph derives creation order and providers translate intent into calls. Nothing executes faster, stateless, or plan-less.

✗ A — speed is not a semantic property

✗ C — state is what makes declaration work

✗ D — planning is the derivation step itself

🎓 EXAM — Declarative = describe the destination, let the machine route.

✓ D2-07 • D2-08 • D3-01

Q-D2-07 ✓ **C** — Anyone can write a provider — plugins over RPC.

💡 **WHY** — Providers are separate processes behind a defined interface; HashiCorp owns some, the community and vendors own hundreds more, and your team can own one for internal APIs.

✗ **A** — the registry alone disproves exclusivity

✗ **B** — no fork needed — that is what the plugin boundary is for

✗ **D** — Sentinel governs policy, not APIs

🎓 **EXAM** — In-house API + "can we manage it" → yes, custom provider.

Q-D2-08 ✓ **D** — Deepest AWS-native integration = CloudFormation.

💡 **WHY** — Single-cloud depth is CloudFormation's entire value proposition: free, native, StackSets, auto-rollback — at the price of every other cloud. The stem asked for depth, not breadth.

✗ **A** — multi-cloud is the opposite of the ask

✗ **B** — AMLs don't integrate CloudFormation deeper

✗ **C** — agents solve configuration, not nativeness

🎓 **EXAM** — Ask what the stem prices: depth → CFN, breadth → Terraform, fleet config → Ansible.

Q-D3-01 ✓ **A** — New clone needs terraform init first.

💡 **WHY** — No plugins, no modules, no backend — nothing works until init downloads and wires them. Plan, console, and fmt all assume init already ran.

✗ **B** — plan needs providers to refresh anything

✗ **C** — console evaluates against installed providers

✗ **D** — fmt works but proves nothing about readiness

🎓 **EXAM** — Empty dir + "first" → init. Always.

✓ D3-02 · D3-03 · D3-04

Q-D3-02 ✓ **B** — .terraform/ holds binaries and is ignored.

💡 WHY — Downloaded plugins are re-fetchable weight: identical bytes return on every init via the lock file. Committing them bloats the repo for zero reproducibility gain.

✗ A — committed binaries are megabytes of regret

✗ C — state is a different file with different rules

✗ D — /usr/bin is outside the project entirely

🎓 EXAM — Binaries ignored, lock committed — the split never inverts.

Q-D3-03 ✓ **C** — -var wins over everything below it.

💡 WHY — The ladder resolves top-down at plan time: CLI flags outrank every file, files outrank env vars, everything outranks defaults. The stray flag is always the prime suspect.

✗ A — defaults lose to literally everything

✗ B — env vars sit BELOW files, not above

✗ D — terraform.tfvars loses to auto overlays and CLI

🎓 EXAM — Value surprise? Strip layers top-down: -var, files, env, default.

Q-D3-04 ✓ **D** — workspace show prints only the active name.

💡 WHY — One word, no metadata — it answers "where am I", not "what exists". Resource lists come from state list; backends from config.

✗ A — regions live in provider blocks

✗ B — list shows names; show shows ONE name

✗ C — buckets live in backend blocks

🎓 EXAM — show = where am I. list = what exists. Never confuse them.

✓ D3-05 • D3-06 • D3-07

Q-D3-05 ✓ **A** — default survives every delete attempt — it is implicit and permanent.

🧠 WHY — Every backend has a default workspace whether you asked for one or not; deleting it would strand unscoped state. Migrate away from it, never remove it.

✗ B/C/D — named workspaces delete freely once empty — only default is special

🎓 EXAM — Delete-default questions answer themselves: the implicit one is immortal.

Q-D3-06 ✓ **B** — terraform.workspace returns the active name for lookups.

🧠 WHY — It is a read-only string, most often the key into per-env maps for sizing, CIDRs, and tags. It never lists, versions, or addresses anything.

✗ A — backends come from blocks, not expressions

✗ C — resources come from state list

✗ D — versions come from the lock file

🎓 EXAM — workspace in code = a string key. Nothing more, exactly enough.

Q-D3-07 ✓ **C** — validate checks syntax and consistency with zero cloud contact.

🧠 WHY — It parses configuration, resolves references, and type-checks — all locally. Anything requiring AWS (refresh, plan diffs, costs) is outside its universe.

✗ A — refresh needs credentials and APIs

✗ B — sandboxes don't exist in Terraform

✗ D — formatting is fmt's job

🎓 EXAM — Validate = syntax gate. No credentials, no network, no excuses for skipping it.

✓ D3-08 • D3-09 • D4-01

Q-D3-08 ✓ **D** — Per-team sizes come from variables, not copies.

🧠 **WHY** — One variable with per-team values (or per-workspace maps) keeps a single codebase honest. Second repos, console edits, and hardcoded locals all fork the truth.

✗ **A** — repos multiply drift

✗ **B** — console edits evaporate at next apply

✗ **C** — hardcoded locals can't vary per caller

🎓 **EXAM** — Vary by input, never by copy. That sentence ends whole interview threads.

Q-D3-09 ✓ **A** — `~> 5.0` means 5.x automatically, 6.0 never.

🧠 **WHY** — Pessimistic pinning takes compatible fixes and refuses breaking majors: automation without 3 AM surprises. Bare `>=` invites the major; exact pins toil on every patch.

✗ **B** — uncapped ranges install breaking majors silently

✗ **C** — exact pins freeze bug-fixes too

✗ **D** — no key floats every release

🎓 **EXAM** — `~>` is the house default. Quote it by expansion: `minor-flex`, `major-freeze`.

Q-D4-01 ✓ **A** — `terraform fmt -check` fails the build on unformatted HCL.

🧠 **WHY** — Exit 1 on any unformatted file, zero writes — the perfect CI gate. Bare `fmt` rewrites (for laptops), `validate` checks syntax (different job), `plan` previews infra (different universe).

✗ **B** — rewrites locally — gates must not mutate

✗ **C** — syntax gate, not style gate

✗ **D** — previews infrastructure, says nothing about style

🎓 **EXAM** — Gate = `-check`. Fix = bare `fmt`. Different verbs, different jobs.

✓ D4-02 · D4-03 · D4-04

Q-D4-02 ✓ **A** — Provider tokens live in credentials.tfrc.json.

🌸 WHY — TFC/TFE API tokens are CLI credentials, not configuration: a home-directory JSON block completely outside the repo. Any repo file (tf, tfstate, lock) would leak secrets to every reader.

✗ B — tfstate holds secrets it shouldn't — never ADD more

✗ C — lock holds hashes, not tokens

✗ D — env vars are the ALTERNATE path, not this file

🎓 EXAM — Token + CLI + file → credentials.tfrc.json. Token + CLI + shell → env vars.

Q-D4-03 ✓ **D** — .terraform.lock.hcl pins EXACT builds for reproducibility.

🌸 WHY — Hashes per platform mean every engineer and CI downloads byte-identical binaries. versions.tf declares ranges; the lock freezes them.

✗ A — ranges declare INTENT, the lock freezes REALITY

✗ B — tfstate versions infra, not plugins

✗ C — outputs export values, pin nothing

🎓 EXAM — Intent (versions.tf) vs frozen reality (lock). The exam lives in that gap.

Q-D4-04 ✓ **B** — echo expression | terraform console tests values without files.

🌸 WHY — Console reads stdin, so piped expressions evaluate with zero project context. Writing a .tf for a one-line check is pure toil.

✗ A — -target runs real changes — absurd for a syntax check

✗ C — refresh mutates state — opposite of safe

✗ D — show reads state — your expression isn't there

🎓 EXAM — One-line check = pipe to console. If you touch a file, you overpaid.

✓ D4-05 • D4-06 • D4-07

Q-D4-05 ✓ **D** — TRACE drowns the log; re-run minimal first.

🧠 WHY — Full TRACE of a big apply is thousands of lines of noise. Minimum reproduction (single resource, lower verbosity) isolates signal before you spend log-reading hours.

✗ A — downgrading past the bug hides the evidence

✗ B — secrets in logs are a DIFFERENT emergency

✗ C — providers log through the same channel — same noise

🎓 EXAM — Big log + weird bug → shrink the blast radius of the reproduction first.

Q-D4-06 ✓ **B** — Format-on-save plus CI -check covers every angle.

🧠 WHY — Prevention at the keyboard, enforcement at the gate: no human discipline required. Either half alone leaks eventually.

✗ A — save-only leaks via web editors and scripts

✗ C — validate checks syntax, not style

✗ D — pre-commit is a THIRD layer, not the pair

🎓 EXAM — Prevention + enforcement: name both halves or the answer is half.

Q-D4-07 ✓ **C** — Keys leave the repo after one scare: env vars and OIDC.

🧠 WHY — Secrets flow through the environment, never through git history. Vaults are the enterprise answer; the repo is never any answer.

✗ A — hardcoded keys are the incident, not the fix

✗ B — tfstate LEAKS secrets — it never stores them safely

✗ D — lock holds hashes — wrong file, wrong job

🎓 EXAM — Keys + repo = the incident. Keys + env/OIDC = the fix.

✓ D4-08 • D5-01 • D5-02

Q-D4-08 ✓ **C** — Wrappers must preserve exit codes and plan-file flow.

💡 WHY — Automation decides (gate, notify, apply) by exit status and saved-plan artifacts. Colors, prompts, and committed .terraform dirs are interactive comforts that pipelines must shed, not keep.

✗ A — committing binaries into automation images is bloat

✗ B — color is the first thing pipes strip

✗ D — prompts hang headless runners forever

🎓 EXAM — Pipeline questions: machine-readable out (codes, files), machine-hostile in (prompts, color).

Q-D5-01 ✓ **D** — Third-party module needs source AND version.

💡 WHY — Source says WHERE, version says WHICH — pinning without a source is a wish, sourcing without a pin is a gamble. Local paths take no version key because there is no registry to resolve.

✗ A — local paths accept no version argument

✗ B — source alone floats every release

✗ C — git refs pin commits, not registry versions

🎓 EXAM — Registry module = two keys or it is wrong. Count the keys in each option.

Q-D5-02 ✓ **B** — VPC module wiring is source, inputs, outputs.

💡 WHY — The contract is three parts: where the code lives, what you feed it, what it hands back. A fourth invented part (credentials inside, state inside) always marks the distractor.

✗ A — extra invented fourth elements expose distractors

✗ C — missing any of the three breaks the contract

✗ D — provider config flows DOWN, never as module input

🎓 EXAM — Three-part contract: source, in, out. Options with two or four parts die first.

✓ D5-03 • D5-04 • D5-05

Q-D5-03 ✓ **C** — Init collision comes from .terraform/modules, never the registry.

💡 WHY — Installed modules cache locally; two copies of one source collide in that directory. The registry is remote and read-only — it cannot collide with anything.

✗ A — the registry serves; it never installs twice

✗ B — state tracks infra, not module code

✗ D — the lock pins providers, not modules

🎓 EXAM — Collision = two local things, one local path. Remote services are never the answer.

Q-D5-04 ✓ **C** — Root calls children; children never call root.

💡 WHY — Composition flows one way: the root wires child modules together via their outputs. A child reaching upward would tangle every reuse — the exam tests this directionality relentlessly.

✗ A — upward calls would make reuse impossible

✗ B — siblings communicate only through the root

✗ D — state files mirror the same hierarchy

🎓 EXAM — Draw the arrow: root → child. Any option reversing it is wrong.

Q-D5-05 ✓ **A** — Init installs; apply only runs installed code.

💡 WHY — Module installation is init's job (download, unpack, record). Apply assumes everything is already on disk — new module code without init is invisible to it.

✗ B — plan reads code; it never fetches it

✗ C — validate parses local files only

✗ D — refresh touches state, not module sources

🎓 EXAM — New module + weird behavior → "did you init?" ends the thread.

✓ D5-06 • D5-07 • D5-08

Q-D5-06 ✓ **D** — Module inputs arrive through variables; outputs leave through outputs.

🧠 **WHY** — Access scope is the whole game: only declared outputs cross the module boundary. Functions compute, locals stay private, state files are never an interface.

✗ **A** — functions transform — they transport nothing

✗ **B** — locals are module-private by design

✗ **C** — remote state reads STACKS, not modules

🎓 **EXAM** — Boundary questions: variables in, outputs out. Everything else stays inside.

Q-D5-07 ✓ **B** — Chef-managed hosts are not module backends.

🧠 **WHY** — Module sources are code locations (local, registry, git, HTTP, S3) — not configuration-management tools. Chef manages machines; it hosts no Terraform code.

✗ **A** — S3 URLs serve modules fine

✗ **C** — git is the canonical private source

✗ **D** — the registry is the default source

🎓 **EXAM** — Source = where CODE lives. Tools that manage MACHINES never qualify.

Q-D5-08 ✓ **A** — Child values surface as normal references.

🧠 **WHY** — module.vpc.vpc_id behaves like any other value once outputs are declared — no special syntax, no data source needed. The boundary is crossed at declaration time, not at use time.

✗ **B** — data sources read PROVIDERS, not child modules

✗ **C** — remote state reads STACKS, not modules

✗ **D** — re-declaring duplicates the boundary

🎓 **EXAM** — If the output is declared, use it like any value. No ceremony.

✓ D6-01 • D6-02 • D6-03

Q-D6-01 ✓ **D** — Write comes before init in the canonical workflow.

💡 **WHY** — Nothing initializes without files to initialize: write → init → plan → apply. Any option starting elsewhere describes a loop, not the lifecycle.

✗ **A** — plan needs initialized providers

✗ **B** — apply needs a plan or code

✗ **C** — init needs SOMETHING to install for

🎓 **EXAM** — First word of the workflow is write. Stems asking "before init" answer themselves.

Q-D6-02 ✓ **B** — Empty plan with zero counts means clean success.

💡 **WHY** — No matching resources = no work = exit 0 with an empty diff. Missing providers, bad syntax, and dead backends all ERROR loudly instead.

✗ **A** — credential failures error, never empty-plan

✗ **C** — syntax failures halt before diffing

✗ **D** — backend failures halt before refreshing

🎓 **EXAM** — Empty diff + exit 0 = healthy no-op. Empty diff + error = broken. Read the exit code.

Q-D6-03 ✓ **C** — Preview-then-gate is the PR workflow.

💡 **WHY** — Plan on every pull request, human review, gated apply: infrastructure gets the same review discipline as application code. Auto-apply on merge skips the review that justifies the pipeline.

✗ **A** — merge-gated apply skips human judgment

✗ **B** — console edits bypass everything

✗ **D** — apply-without-plan is the anti-pattern

🎓 **EXAM** — PR + plan + gate: the three words of every workflow question.

✓ D6-04 • D6-05 • D6-06

Q-D6-04 ✓ **C** — Aborting mid-apply leaves partial real resources.

🌸 WHY — Terraform stops issuing new calls, but finished creates stand — the next plan reconciles them. "All-or-nothing rollback" is the fiction; the plan is the recovery tool.

✗ A — completed API calls don't un-happen

✗ B — state is written incrementally — it never waits

✗ D — retries re-enter through a fresh plan, not a resume

🎓 EXAM — Ctrl-C question? Answer: partial stands, next plan heals. Never promise rollback.

Q-D6-05 ✓ **A** — Saved plans bind reviewed code to the apply.

🌸 WHY — apply plan.out executes EXACTLY the reviewed diff — no drift window, no re-evaluation. Re-planning at apply time re-opens the gap the saved plan closed.

✗ B — re-planning re-opens the drift window

✗ C — auto-approve skips the review, not the binding

✗ D — console edits bypass the pipeline entirely

🎓 EXAM — Reviewed plan + saved file = what you saw is what runs.

Q-D6-06 ✓ **A** — destroy -out saves a teardown preview.

🌸 WHY — The destroy plan serializes like any other: review the death list, then apply the file. -target limits scope (different job); auto-approve skips review (opposite job).

✗ B — -target narrows; -out SERIALIZES

✗ C — auto-approve is the anti-review

✗ D — -refresh-only previews drift, not teardown

🎓 EXAM — -out = serialize any plan, including destroy. One flag, every direction.

✓ D6-07 • D6-08 • D6-09

Q-D6-07 ✓ **A** — Registry version bumps install via init.

🧠 WHY — New module code enters through init (fresh or -upgrade); apply never fetches. Editing the lock by hand fights the tool that owns it.

✗ B — apply runs code; it never downloads it

✗ C — the lock RECORDS installs, it doesn't perform them

✗ D — hand-editing tool-owned files corrupts them

🎓 EXAM — New code in → init. Every time, no exceptions.

Q-D6-08 ✓ **D** — The apply failed — resources may or may not exist.

🧠 WHY — Exit 1 mid-apply guarantees nothing about the world: inspect state and reality, then let the next plan reconcile. "All rolled back" and "all applied" are both fictions.

✗ A — no rollback exists to complete

✗ B — partial creates stand until reconciled

✗ C — the lock releases; the mess remains

🎓 EXAM — Failed apply → inspect, never assume. The next plan is the recovery tool.

Q-D6-09 ✓ **B** — apply -refresh-only syncs state toward reality deliberately.

🧠 WHY — Explicit reconciliation mode: real-world truth flows into state with intent, unlike incidental refresh during plan. Default plans propose changes; destroy previews teardown; validate checks syntax.

✗ A — default plans propose; they don't deliberately sync

✗ C — destroy previews teardown, not truth

✗ D — validate never touches state

🎓 EXAM — Deliberate sync = refresh-only. Incidental refresh rides along.

✓ D7-01 • D7-02 • D7-03

Q-D7-01 ✓ **A** — State maps config to reality: the binding record.

🌸 WHY — Plan and apply are both joins against this table — without it Terraform cannot tell managed from foreign, desired from actual. Logs, caches, and binaries do none of this.

✗ B — logs narrate; they bind nothing

✗ C — .terraform caches CODE, not bindings

✗ D — binaries execute; they remember nothing

🎓 EXAM — Mapping question → the JSON binding file. Every time.

Q-D7-02 ✓ **B** — Drift = reality moved; plan detects, apply cures.

🌸 WHY — Hands in the console (or outside automation) desync the triangle: config ≠ state ≠ real. Refresh surfaces it as a diff; apply re-asserts code as truth.

✗ A — code edits are INTENT, not drift

✗ C — lock failures block runs; they move nothing

✗ D — version bumps change providers, not reality

🎓 EXAM — Drift trilogy: console cause, plan detection, apply cure. Quote all three.

Q-D7-03 ✓ **C** — Backend block lives INSIDE terraform {}.

🌸 WHY — Backend is a nested block of the terraform settings block — provider blocks configure APIs, variables take inputs, outputs export values. The exam loves this nesting.

✗ A — providers talk to CLOUDS, not state

✗ B — variables take inputs — any type

✗ D — outputs export — they store nothing

🎓 EXAM — State config nests in terraform {}. Say the nesting, not just the name.

D7-04 • D7-05 • D7-06

Q-D7-04  **D** — S3 + DynamoDB: bucket holds state, table holds the lock.

 **WHY** — Two AWS services, two jobs: versioned encrypted storage plus conditional-write locking. Either half alone is half a backend.

 **A** — local paths have no locking at all

 **B** — git versions code — state needs a lock git lacks

 **C** — the registry serves modules, not locks

 **EXAM** — Name BOTH halves and each half's job. Half answers get half credit.

Q-D7-05  **D** — Crashed holder? Confirm death, then force-unlock.

 **WHY** — Locks are protective, not punitive: verify no live process holds it (stale crash), then release by ID. Deleting tables or restoring backups for a lock is surgery for a splinter.

 **A** — deleting the table destroys the locking for EVERYONE

 **B** — backups restore STATE — the lock isn't state

 **C** — waiting on a dead process waits forever

 **EXAM** — Lock drill: confirm dead → force-unlock by ID. Two steps, in order.

Q-D7-06  **A** — state rm stops tracking; the resource stands.

 **WHY** — Surgery on the binding, not the world: Terraform forgets, the cloud keeps billing. Destroy first when you mean destroy; rm when you mean abandon-to-other-management.

 **B** — nothing is deleted — that's the danger AND the feature

 **C** — rm edits STATE, never configuration

 **D** — providers are untouched by state surgery

 **EXAM** — rm = forget, not destroy. The cloud keeps the resource AND the bill.

✓ D7-07 • D7-08 • D7-09

Q-D7-07 ✓ **A** — Rename without rebirth = terraform state mv.

💡 WHY — The binding moves, the resource never notices: new address, same cloud object, empty plan. Taint/replace destroys; rm abandons; mv renames.

✗ B — taint schedules DESTRUCTION — opposite of the ask

✗ C — rm forgets; the new address would RECREATE

✗ D — apply without mv plans a delete+create pair

🎓 EXAM — Rename = mv. Destroy = taint. Forget = rm. Three verbs, three fates.

Q-D7-08 ✓ **D** — Lock fights mean a second writer — coordinate, don't force.

💡 WHY — Concurrent applies on one state need human sequencing, not lock-breaking: force-unlock under a live apply corrupts state for both runners.

✗ A — force under a LIVE holder corrupts — the cardinal sin

✗ B — retries collide again without coordination

✗ C — the table is healthy; the PROCESS is the conflict

🎓 EXAM — Live lock + force = corruption. Wait, queue, serialize. Force is for the DEAD only.

Q-D7-09 ✓ **B** — Two states, one name: duplicate resources, split brain.

💡 WHY — Local files don't see each other — each apply births a parallel world under the same names. Locking exists precisely because local state can't do this check.

✗ A — clean no-op requires ONE shared state

✗ C — locks are a REMOTE feature — locals have none

✗ D — providers happily create whatever they're asked

🎓 EXAM — Two writers + no lock = duplicates. The sentence that justifies every backend.

**D7-10 • D7-11 • D7-12****Q-D7-10** **C** — Migrate with dual backends declared, then init -migrate-state. **WHY** — Both blocks present lets Terraform copy, verify, and flip the pointer in one guided move. Hand-copying JSON or deleting locals invites split-brain. **A** — hand JSON copies skip verification **B** — deleting local state strands history **D** — apply recreates — migration must move, not rebuild **EXAM** — Migration = declare both → init migrates. Never hand-carry state JSON.**Q-D7-11** **C** — Snapshot raw state BEFORE surgery, verify after. **WHY** — State mv/rm/import rewrite the binding blind: a versioned backup turns mistakes into restores. Planning after (not before) is what confirms the surgery. **A** — planning first tells you nothing about the edit **B** — locks guard CONCURRENCY, not your typos **D** — backups without verification restore blindly **EXAM** — Surgery rule: snapshot, cut, plan-verify. Skip step one, own step four.**Q-D7-12** **D** — Versioning + encryption + locking = the backend checklist. **WHY** — Every option alone is necessary and insufficient: versions for history, encryption for secrets, locking for teams. The exam's "BEST" questions demand the complete set. **A/B/C** — each names ONE pillar and omits two — partial answers **EXAM** — BEST-backend questions: count the pillars. One pillar = wrong. Three = right.

✓ D7-13 • D7-14 • D7-15

Q-D7-13 ✓ **B** — state replace-provider rewrites every binding at once.

💡 WHY — One command re-points all provider addresses in the binding — no per-resource surgery, no state deletion, no recreation. It exists precisely for fork migrations.

✗ A — per-resource addressing doesn't scale to whole configs

✗ C — deleting state strands every binding

✗ D — re-apply recreates what should only be re-pointed

🎓 EXAM — Whole-provider moves = one rewrite command. Per-resource tools are for per-resource jobs.

Q-D7-14 ✓ **B** — taint marks for replace; the NEXT plan shows the pair.

💡 WHY — Taint writes intent into state; destruction happens at apply, previewed as -/+ in the plan. Dirty reads need refresh, not taint; immediate destruction skips the preview.

✗ A — nothing dies until apply runs

✗ C — taint marks STATE, never configuration

✗ D — imports ADD bindings; taints schedule rebirths

🎓 EXAM — Taint = schedule. Plan = preview (-/+). Apply = execute. Three beats, in order.

Q-D7-15 ✓ **C** — Clean plan, empty state list = apply targeted elsewhere.

💡 WHY — An empty list with a clean diff means the resources live in ANOTHER state (wrong workspace, wrong backend, -target elsewhere) — not that they don't exist.

✗ A — failed applies ERROR — they don't go quiet

✗ B — console edits show as DRIFT, not absence

✗ D — locks block runs; they hide nothing

🎓 EXAM — Clean + empty = looking in the wrong state. Check workspace and backend first.

✓ D7-16 • D7-17 • D7-18

Q-D7-16 ✓ **D** — Refresh-only previews drift without touching anything.

🧠 WHY — Read-only reconciliation: real-world truth flows into a diff you can inspect, adopt, or revert — zero writes to state or cloud.

Destructive options for a read-only question are always wrong.

✗ A — reverting destroys evidence before review

✗ B — force-unlock fixes LOCKS, not drift

✗ C — tainting schedules rebirth for a reading job

🎓 EXAM — Drift review = read-only preview first. Hands off the destroy flags.

Q-D7-17 ✓ **B** — Sensitive values leak through DISPLAY, not storage.

🧠 WHY — State holds secrets regardless; sensitive=true only masks console and log output. Encryption protects storage (different layer); deletion fantasies ignore reality.

✗ A — nothing encrypts just by flagging

✗ C — outputs still carry the value downstream

✗ D — state keeps the plaintext either way

🎓 EXAM — sensitive = display mask. Encryption = storage guard. Two layers, two tools.

Q-D7-18 ✓ **A** — Ephemeral values never touch state.

🧠 WHY — Write-only arguments and ephemeral blocks stay in memory: plans show "(ephemeral)", providers consume, nothing persists. Every other option stores or leaks.

✗ B — sensitive still STORES — it only masks display

✗ C — outputs persist by definition

✗ D — locals live in state-adjacent config

🎓 EXAM — Must-not-persist → ephemeral. The exam's newest favorite word.

✓ D8-01 • D8-02 • D8-03

Q-D8-01 ✓ **A** — Variables parametrize; locals compute; outputs export.

💡 WHY — Three blocks, three jobs: inputs vary per caller, locals derive within a module, outputs cross boundaries. Mixing their jobs is the exam's favorite confusion.

✗ B — locals can't take caller input

✗ C — outputs can't feed the same module

✗ D — resources create — they parametrize nothing

🎓 EXAM — Vary, derive, export: say which job before naming the block.

Q-D8-02 ✓ **B** — Objects group related fields into one typed value.

💡 WHY — One variable carrying name, size, and tags beats three loose strings drifting apart. Maps alone lose per-field types; splitting multiplies wiring.

✗ A — separate variables drift apart

✗ C — maps allow any shape — objects enforce it

✗ D — locals derive; they don't take input

🎓 EXAM — Related fields travel together: object type, one variable.

Q-D8-03 ✓ **C** — Validation rejects bad input at plan time.

💡 WHY — Rules on variables fail fast with custom messages — before any API call, before any cost. Documentation comments never enforce; apply-time errors bill first.

✗ A — comments describe; they enforce nothing

✗ B — post-apply errors already cost money

✗ D — sensitive masks display — validates nothing

🎓 EXAM — Reject early = validation block. The exam rewards fail-fast everywhere.

✓ D8-04 • D8-05 • D8-06

Q-D8-04 ✓ **D** — One map + for_each beats N nearly-identical blocks.

🧠 **WHY** — Data varies, logic stays single: add a map entry, get a subnet. Count-indexed copies and duplicated blocks fork the logic with every edit.

✗ **A** — duplicated blocks fork on every future edit

✗ **B** — count breaks on middle removals

✗ **C** — workspaces isolate ENVs, not subnets

🎓 **EXAM** — Repeated blocks with varying data = for_each over a map. Reflex-level.

Q-D8-05 ✓ **A** — Dynamic blocks generate repeated nested blocks.

🧠 **WHY** — Ingress rules, tags, and lifecycle stanzas repeat **INSIDE** a resource — dynamic + for_each stamps them from data. Splat **READS** collections; count clones **WHOLE** resources.

✗ **B** — splat reads; it never generates

✗ **C** — count clones resources, not nested blocks

✗ **D** — workspaces isolate environments

🎓 **EXAM** — Nested repetition = dynamic. Whole-resource repetition = count/for_each.

Q-D8-06 ✓ **A** — N ingress blocks generate from a list variable.

🧠 **WHY** — One dynamic block over a list of rule objects: data in, stanzas out. count on the **RESOURCE** would clone whole security groups — wrong layer entirely.

✗ **B** — resource-level count clones the whole SG

✗ **C** — hardcoded rules can't vary per caller

✗ **D** — outputs export — they generate nothing

🎓 **EXAM** — Check the layer: nested block → dynamic. Whole resource → count/for_each.

✓ D8-07 • D8-08 • D8-09

Q-D8-07 ✓ **C** — Data sources READ; they never create.

💡 WHY — AMIs, existing VPCs, current regions: data blocks query and expose, adding zero resources to the plan. Managed resources create; outputs export; variables input.

✗ A — resources CREATE — the opposite job

✗ B — outputs export values downstream

✗ D — variables carry input, query nothing

🎓 EXAM — Read-only lookup = data source. "Latest AMI" is the tell.

Q-D8-08 ✓ **B** — Orphaned t2.micro + -target: the classic scoped-apply scar.

💡 WHY — Targeted applies skip everything outside scope — un-targeted resources drift from code silently. The fix is a full apply, not more targeting.

✗ A — deleting state hides the symptom

✗ C — more -target narrows further — wrong direction

✗ D — locks guard concurrency, not scope

🎓 EXAM — -target scars: untargeted resources drift silently. Full apply heals.

Q-D8-09 ✓ **D** — for_each keys must be stable and unique.

💡 WHY — Keys are identity: renames rebirth, duplicates collide, indices shift. Stable natural keys (names, AZs) survive edits that destroy count-based addressing.

✗ A — indices shift on middle removal — the count trap

✗ B — duplicates collide at plan time

✗ C — renames are destroy+create pairs

🎓 EXAM — Keys = identity. Stable, unique, rename-proof. Three adjectives or wrong.

✓ D8-10 • D8-11 • D8-12

Q-D8-10 ✓ **B** — Count removal shifts every later index.

💡 WHY — Index is identity under count: deleting [1] renames [2]→[1], forcing rebirth down the line. `for_each` keys don't shift — the exam's core repetition lesson.

✗ A — nothing is free — later resources rebirth

✗ C — first-element removal shifts EVERYTHING

✗ D — state edits can't fix index identity

🎓 EXAM — Middle-out under count = cascade. The sentence that sells `for_each`.

Q-D8-11 ✓ **C** — Splatting collects one attribute across instances.

💡 WHY — `aws_instance.web[*].public_ip` projects the fleet into a list: outputs, security rules, and inventories all consume it. `for_each` BUILDS; `dynamic` GENERATES; `depends_on` ORDERS.

✗ A — `for_each` builds resources, not lists

✗ B — `dynamic` generates blocks

✗ D — `depends_on` orders creation

🎓 EXAM — Collect-across = splat. Build-many = `for_each`. Generate-nested = `dynamic`.

Q-D8-12 ✓ **D** — Unknown values defer: plan shows (known after apply).

💡 WHY — Anything derived from not-yet-created resources is unknowable until apply: Terraform marks it, plans around it, resolves later. Errors, nulls, and skips would all lie.

✗ A — unknowns aren't errors — they're futures

✗ B — null would corrupt downstream joins

✗ C — the plan continues around the unknown

🎓 EXAM — (known after apply) = patience, not failure. Read it as a promise.

✓ D8-13 · D8-14 · D8-15

Q-D8-13 ✓ **A** — 5 subnets from a /16 need newbits = 3 (8 slots).

🌸 WHY — $2^3 = 8 \geq 5$; newbits 2 gives only 4. Round UP to the next power of two.

✗ B — 4 slots can't hold 5 subnets

✗ C — newbits counts BITS, not subnets

✗ D — 8 newbits carves 256 /24s — overkill

🎓 EXAM — Slots = $2^{\text{newbits}} \geq \text{needed}$. Round up, always.

Q-D8-14 ✓ **B** — merge() collisions: right side wins.

🌸 WHY — Later maps override earlier ones — the layering primitive. Errors, left-wins, and both-kept all misremember.

✗ A — no error — deterministic override

✗ C — left loses by design

✗ D — one key holds one value

🎓 EXAM — Layering order = argument order. Last writer wins.

Q-D8-15 ✓ **C** — try() guards optional nested access without crashing.

🌸 WHY — First-success-wins with fallback: missing paths return the default instead of erroring. Encryption, speed, and deletion are wrong universes.

✗ A — no encryption anywhere

✗ B — errors become fallbacks — not speed

✗ D — resources are untouched by reads

🎓 EXAM — Optional access = try(). Missing path = fallback, not failure.

✓ D9-02 • D9-03 • D9-04

Q-D9-02 ✓ **B** — Workspaces isolate STATE, never code or credentials.

🧠 WHY — Same code, N state files: dev and prod differ only in their bindings. Separate repos fork code; separate backends fork truth; shared credentials fork risk.

✗ A — code stays single — that's the point

✗ C — backends stay shared — states differ inside

✗ D — credentials scope separately

🎓 EXAM — Workspace = state fork. Everything else stays shared. One sentence.

Q-D9-03 ✓ **C** — State surgery verbs: mv renames, rm forgets, import adopts.

🧠 WHY — Three verbs, three fates: rename the binding, drop the binding, create a binding. Apply creates RESOURCES — never bindings.

✗ A — apply manages resources, not bindings

✗ B — init installs code, touches no bindings

✗ D — refresh reads reality, rewrites nothing

🎓 EXAM — Binding verbs: mv/rm/import. Resource verbs: plan/apply. Never cross them.

Q-D9-04 ✓ **D** — Splat over for_each instances feeds module outputs.

🧠 WHY — module.instances[*].private_ip collects across keyed instances into the app module's input: collect-then-pass. Count-indexing keyed resources fails; locals can't see inside modules.

✗ A — string keys don't index numerically

✗ B — locals can't pierce module boundaries

✗ C — remote state reads STACKS, not sibling modules

🎓 EXAM — Collect with splat, pass as input. The composition one-liner.

✓ D9-05 • D9-06 • D9-07

Q-D9-05 ✓ **A** — HCP Terraform runs: remote execution, shared state, governance.

💡 WHY — The value stack is runs + collaboration + policy: speculative plans on PRs, locked shared state, Sentinel guardrails. Local runs with a remote backend get storage without any of the platform.

✗ B — storage without runs is just a backend

✗ C — local runs keep local blast radius

✗ D — CLI workspaces isolate state — nothing more

🎓 EXAM — Platform = runs + sharing + policy. Backend = storage. Price the difference.

Q-D9-06 ✓ **B** — Speculative plans preview PRs with zero state risk.

💡 WHY — Dry-run diffs on every pull request: reviewers see infra impact before merge, state untouched, nothing applied. Gates without previews approve blind.

✗ A — blind auto-apply is the anti-pattern

✗ C — locks serialize — they preview nothing

✗ D — local plans can't comment on PRs

🎓 EXAM — PR + preview + no-touch = speculative. The governance flywheel.

Q-D9-07 ✓ **C** — Sentinel GOVERNS with policy-as-code.

💡 WHY — Mandatory guardrails evaluated at plan time: regions, sizes, tags — violations fail the run before apply. Cost tools estimate; workspaces isolate; runs execute.

✗ A — cost estimation PRICES — Sentinel PERMITS

✗ B — workspaces isolate state, enforce nothing

✗ D — runs execute; policy judges

🎓 EXAM — Sentinel = policy judge. Cost tool = price tag. Different sentences.

✓ D8-16 • D9-08 • D9-01

Q-D8-16 ✓ **A** — Validation runs at plan, rejecting bad input with your message.

🧠 WHY — Rules on variables fail fast with custom errors — before any API call, before any cost. Decorative-never and console-only misunderstand the feature; after-apply is already too late.

✗ B — decorative-never denies the feature exists

✗ C — console-only shrinks a plan-time gate

✗ D — after apply the money is spent

🎓 EXAM — Reject early = validation block. Fail-fast wins.

Q-D9-08 ✓ **D** — Agents run where your cloud lives; the platform orchestrates.

🧠 WHY — Hybrid split: private network execution, SaaS control plane. All-local loses governance; all-remote can't reach private subnets; workspaces isolate state, not networks.

✗ A — pure local keeps local blast radius

✗ B — pure remote can't touch private nets

✗ C — workspaces isolate STATE, not networks

🎓 EXAM — Private subnets + SaaS control = agents. The hybrid sentence.

Q-D9-01 ✓ **A** — HCP lineage: TFC managed, TFE self-hosted.

🧠 WHY — Same platform, different operators: SaaS vs your basement. The exam tests who patches, who scales, who holds the keys.

✗ B — self-hosted is TFE's definition

✗ C — agents are a NETWORK feature, not a tier

✗ D — workspaces exist in both

🎓 EXAM — Who operates it? SaaS = TFC, self-hosted = TFE.

 M1-01 → M1-04

Q-M1-01  **A** — fmt -check gates CI without rewriting.

 WHY — Gate = -check (exit 1, no writes); bare fmt is the local fixer. Rewriting in CI mutates what it judges.

Q-M1-02  **B** — State maps configuration to real-world objects.

 WHY — The binding table plan/apply join against. Compile-faster/skip-init/encrypt are all wrong-layer fictions.

Q-M1-03  **C** — 0/0/0 on re-apply = idempotence.

 WHY — Same code, same world, no-op second run. Syntax, backends, and reviews are different properties.

Q-M1-04  **D** — VPC ID + CIDR land in state as resource metadata.

 WHY — Apply records what the APIs returned. Re-queried-every-second denies state; versions.tf pins providers.

✓ M1-05 → M1-08

Q-M1-05 ✓ **A** — count 3→2 destroys web[2], the tail.

💡 WHY — Index is identity: the highest index vanishes. Survivors keep addresses; nothing else rebirths.

Q-M1-06 ✓ **B** — default is the undeletable workspace.

💡 WHY — Implicit and permanent — migrate away, never remove. Named envs delete freely once empty.

Q-M1-07 ✓ **C** — for_each takes a set or map.

💡 WHY — Keys need stability: lists re-index, numbers/strings key nothing. Bare lists wrap with toset().

Q-M1-08 ✓ **D** — TFC vs TFE = hosting: SaaS vs self-hosted.

💡 WHY — Same platform, different operator. State format, language, and CLI are identical.

✓ M1-09 → M1-12

Q-M1-09 ✓ **A** — No-override tag policy = mandatory.

💡 WHY — Mandatory hard-blocks; soft-mandatory permits override; advisory logs; disabled enforces nothing.

Q-M1-10 ✓ **B** — Login tokens live in credentials.tfrc.json.

💡 WHY — Home-dir CLI credentials, outside any repo. tfstate/lock/S3 are all wrong-layer or leak-prone.

Q-M1-11 ✓ **C** — Cross-region block needs provider = aws.eu.

💡 WHY — Alias on the block selects the non-default config. Second installs, depends_on, and second states are all wrong tools.

Q-M1-12 ✓ **D** — TF_VAR vs file vs -var: -var wins.

💡 WHY — CLI flags outrank every file; files outrank env; env outranks defaults. Strip top-down.

✓ M1-13 → M1-16

Q-M1-13 ✓ **A** — Deleted state + apply = duplicate resources.

💡 WHY — Terraform forgot everything and recreates the world. No auto-restore, no no-op, no speedup.

Q-M1-14 ✓ **B** — Single-cloud deepest-native = CloudFormation.

💡 WHY — Depth is CFN's whole proposition. Terraform = breadth, Ansible = fleet config, Packer = images.

Q-M1-15 ✓ **C** — Registry modules pin version = "x.y.z".

💡 WHY — Floating latest breaks future runs; branches aren't versions. Pin every registry source.

Q-M1-16 ✓ **D** — PR → auto plan posted = VCS-driven runs.

💡 WHY — Remote runs on VCS events with PR comments. Console mode, local-exec, and state mv are unrelated.

✓ M1-17 → M1-20

Q-M1-17 ✓ **A** — Users+Nginx on live VMs = Ansible.

💡 WHY — Fleet configuration of existing hosts is the config-management layer. Provisioners create; they don't babysit.

Q-M1-18 ✓ **C** — Lock error → read the lock info first.

💡 WHY — Who, what op, since when — then decide stale vs live. Deleting rows and destroying everything are panic moves.

Q-M1-19 ✓ **B** — Workflow order: write, init, plan, apply.

💡 WHY — Files before init, init before plan, plan before apply. Any rotation describes a loop, not the lifecycle.

Q-M1-20 ✓ **D** — String-key addressing implies for_each keys.

💡 WHY — ["us-east-1a"] is a map/set key, never a count index. Count addresses numerically.

 M1-21 → M1-24

Q-M1-21  **A** — Same word, different concept = workspace.

 **WHY** — CLI workspaces isolate state; TFC workspaces bind config+vars+state+runs. The exam's favorite homonym.

Q-M1-22  **B** — Possibly-empty list reads via `one(x[*].attr)`.

 **WHY** — `one()` returns the single element or null — `x[0]` crashes on empty. `count.index` writes, never reads.

Q-M1-23  **C** — Saved plans bind reviewed == executed.

 **WHY** — Serializing the exact diff closes the drift window between review and apply. Re-planning re-opens it.

Q-M1-24  **D** — Declarative = state end-state, tool derives steps.

 **WHY** — WHAT not HOW: graph orders, providers translate. Scripting calls and manual ordering are imperative.

 M1-25 → M1-28

Q-M1-25  **A** — Remote backends: shared truth, locking, secrets off laptops.

 **WHY** — Collaboration needs all three. No-locking/no-backend claims deny the feature; code is never encrypted.

Q-M1-26  **B** — PR cost estimates come from remote-run plan analysis.

 **WHY** — The platform prices the diff. fmt styles, state mv rebinds, consoles click — none price anything.

Q-M1-27  **C** — References create graph edges ordering creation.

 **WHY** — Implicit dependencies from interpolation. No cost, no duplicates — just ordering.

Q-M1-28  **D** — workspace show prints the active name only.

 **WHY** — One word answering "where am I". Resources come from state list; types from config.

✓ M1-29 → M1-32

Q-M1-29 ✓ **A** — Console tagging surfaces as a ~ drift change.

💡 WHY — Refresh imports reality into the diff; clean applies never hide hand edits. Absence claims deny refresh.

Q-M1-30 ✓ **B** — TFC workspaces bind config, variables, state, run history.

💡 WHY — A full execution context, not a key suffix. Suffix-only describes CLI workspaces — the homonym trap.

Q-M1-31 ✓ **C** — ~> 5.0 allows 5.x, never 6.0.

💡 WHY — Minor-flex, major-freeze. Bare >= invites breaking majors; exact pins toil on patches.

Q-M1-32 ✓ **D** — init never creates real infrastructure.

💡 WHY — Downloads, backend config, lock file — all local. Creation starts at apply.

✓ M1-33 → M1-36

Q-M1-33 ✓ **A** — Soft vs hard mandatory = override possibility.

💡 WHY — Permitted-with-rights vs never. Language, speed, and nothing are non-answers.

Q-M1-34 ✓ **B** — default survives every delete.

💡 WHY — Implicit and permanent. Named envs delete freely once empty.

Q-M1-35 ✓ **C** — Noisiest TF_LOG level = TRACE.

💡 WHY — Full RPC detail for provider bugs. OFF silences, INFO summarizes, WARN filters to warnings.

Q-M1-36 ✓ **D** — State secrets are plaintext — restrict access.

💡 WHY — No default encryption, no hashing, never absent. Bucket policy + IAM + Vault/SSM references.

✓ M1-37 → M1-40

Q-M1-37 ✓ **A** — Children receive values via root input variables.

💡 WHY — The module call's arguments are the only door in. Console edits, hardcoded locals, and surgery bypass the contract.

Q-M1-38 ✓ **B** — Pulumi = general-purpose languages vs HCL.

💡 WHY — State, planning, and multi-cloud exist on both sides. Language is the axis; the rest are distractors.

Q-M1-39 ✓ **C** — Local/registry/git/HTTP/S3 = module source types.

💡 WHY — Where code lives. Backends store state, log levels verbosity, workspaces isolate — different lists.

Q-M1-40 ✓ **D** — `{ for k,i : k => i.public_ip }` builds a name→IP map.

💡 WHY — Object comprehension keyed by name. Lists come from splat; subnets and errors are wrong shapes.

✓ M1-41 → M1-44

Q-M1-41 ✓ **A** — validate = syntax + consistency, no cloud.

💡 WHY — Local gate: parses, resolves, type-checks. Costs, locks, and DNS all need a network validate never touches.

Q-M1-42 ✓ **B** — Golden rule: never hand-edit state; use state commands.

💡 WHY — Every fix has a command; hand JSON edits corrupt. Commit/email options violate every other rule too.

Q-M1-43 ✓ **C** — TF_VAR_tags maps need single-quoted HCL.

💡 WHY — Shell-safe quoting around object syntax. Bare braces split, double quotes interpolate, files are another path.

Q-M1-44 ✓ **D** — state pull = raw JSON snapshot before surgery.

💡 WHY — Versioned backup enabling restore. Applies, formatting, and upgrades are unrelated verbs.

✓ M1-45 → M1-48

Q-M1-45 ✓ **A** — Private registry = blessed org modules/providers.
💡 WHY — Versioned, reviewed, internal. CPUs, free state, and no-init are fantasy features.

Q-M1-46 ✓ **B** — output_changes gates deploys on output deltas.
💡 WHY — Machine-readable contract diff for pipelines. Formatting, billing, and DNS are other tools' jobs.

Q-M1-47 ✓ **C** — TFC agents run remote operations on your infra.
💡 WHY — Private-net execution, SaaS control. Laptops, nothing, and fmt-only miss the hybrid point.

Q-M1-48 ✓ **D** — merge() collisions: right side wins.
💡 WHY — Later arguments override. Errors, left-wins, and drops are all wrong mental models.

 M1-49 → M1-52

Q-M1-49  **A** — Ctrl+C leaves partial state; re-apply continues.

 WHY — Finished creates stand recorded; the next plan reconciles. No rollback exists anywhere.

Q-M1-50  **B** — Validation runs at plan, rejecting bad input.

 WHY — Fail-fast with custom messages before cost. After-apply/console/never all miss the gate.

Q-M1-51  **C** — -auto-approve belongs in gated pipelines only.

 WHY — Reviewed plan + automation gate. Laptops, everywhere, and nowhere are all wrong scopes.

Q-M1-52  **D** — workspace new on a taken name: select it instead.

 WHY — Existence error, not corruption. Init, backends, and state are unrelated to naming.

 M1-53 → M1-57

Q-M1-53  **A** — VCS integration posts plans back on the PR.

 **WHY** — Review where code lives. Email, state, and nowhere break the review loop.

Q-M1-54  **B** — `depends_on` covers invisible side-effects only.

 **WHY** — References first, always. Everywhere-clutter and nowhere-purism both break real configs.

Q-M1-55  **B** — console = REPL for expressions, no cloud.

 **WHY** — Pipe values, test functions, touch nothing. Backends, deploys, and registries are other verbs.

Q-M1-56  **D** — force-unlock needs the lock ID after proving death.

 **WHY** — Confirm no live holder, release by ID, verify health. Nothing-less corrupts; buckets are unrelated.

Q-M1-57  **A** — Free-tier caps push to paid tier or scoped usage.

 **WHY** — Scale honestly: sharing tokens, committing state, and ignoring limits all violate other rules.

✓ M2-01 → M2-04

Q-M2-01 ✓ **A** — State never compiles providers.

💡 WHY — EXCEPT-format: mapping, metadata, and plan-cache are real purposes; compiling is the odd one out.

Q-M2-02 ✓ **B** — Max provider logs: TF_LOG=TRACE redirected to file.

💡 WHY — TRACE is loudest; stderr redirect captures. OFF silences; fmt/styles touch nothing.

Q-M2-03 ✓ **C** — Same model, same env, every time = idempotence via IaC.

💡 WHY — Repeatability from versioned declaration. Snowflakes, clicks, and untracked change are the disease.

Q-M2-04 ✓ **D** — Heavy teams outgrow free tier: paid or scoped usage.

💡 WHY — Local-only, committed state, and disabled plans all trade away the platform's value.

✓ M2-05 → M2-08

Q-M2-05 ✓ **A** — web["blue"] implies for_each keys.

💡 WHY — String-key addressing is map/set identity. Count indexes numerically; backends/provisioners are unrelated.

Q-M2-06 ✓ **A** — Pulumi differs: general-purpose languages vs HCL.

💡 WHY — Both keep state, plan, and span clouds. Language is the axis — the rest are fictions.

Q-M2-07 ✓ **C** — Safe bug-fix pin: version = "~> 2.0".

💡 WHY — Pessimistic constraint takes patches, refuses majors. Latest-branches and open ranges gamble.

Q-M2-08 ✓ **D** — TF_VAR_count="three" for number = type error at plan.

💡 WHY — No auto-correction, no silent ignore — the type system fails fast. Destruction claims are panic.

✓ M2-09 → M2-12

Q-M2-09 ✓ **B** — Skipping plan risks unreviewed changes executing blind.

💡 WHY — Plan is the review artifact. Faster-runs and better-state claims skip the safety the question asks about.

Q-M2-10 ✓ **A** — Change without PR = unreviewed, violates peer review.

💡 WHY — IaC's review discipline is the point. Faster/required/encrypted all miss the governance ask.

Q-M2-11 ✓ **C** — Deleted lock table = race without serialization; restore it.

💡 WHY — No lock, no mutual exclusion. Faster/encrypt/skip-init all misread the failure.

Q-M2-12 ✓ **D** — output -json piped to jq serves CI/script consumption.

💡 WHY — Machine-readable exports for automation. Formatting, init, and locking are other verbs.

✓ M2-13 → M2-16

Q-M2-13 ✓ **A** — Speculative PR plans come from VCS-driven runs.

💡 WHY — Remote plans on pull requests, zero state risk. local-exec, fmt, and console are unrelated.

Q-M2-14 ✓ **B** — Red-hour lock: confirm death, force-unlock by ID, prove health.

💡 WHY — Stale-crash drill in order. Waiting forever stalls; bucket deletes and laptop reboots miss the object.

Q-M2-15 ✓ **C** — Private registry hosts blessed org versions.

💡 WHY — Reviewed, versioned, internal modules. Avoiding-init and faster-downloads are fantasies.

Q-M2-16 ✓ **D** — `values(map)[*].id` returns all IDs as a list.

💡 WHY — Values projection then splat. One-ID, map-shape, and nothing misread the two-step.

 M2-17 → M2-20

Q-M2-17  **A** — workspace list shows all, * marks active.

 **WHY** — Inventory plus position. Current-only, counts, and costs are other commands' jobs.

Q-M2-18  **B** — Advisory violation logs; the run continues.

 **WHY** — Informational by design. Blocks belong to mandatory; deletes and bills are fictions.

Q-M2-19  **C** — lookup() with default returns the fallback on miss.

 **WHY** — Three-arg safety net for per-region maps. Crashes need two-arg form; deletes never happen.

Q-M2-20  **D** — Console SG edits surface as ~ changes to reconcile.

 **WHY** — Refresh imports reality; plan proposes code-as-truth. Invisible-error-new-workspace all deny refresh.

 M2-21 → M2-24

Q-M2-21  **A** — apply tfplan guarantees reviewed == executed.

 **WHY** — The exact serialized diff runs. No-state/no-backend claims deny the file; init speed is unrelated.

Q-M2-22  **B** — state show displays one resource's attributes.

 **WHY** — Single-address inspection. Deletes, formats, and workspace creation are other verbs.

Q-M2-23  **C** — Mandatory policy with missing tags hard-blocks.

 **WHY** — No override, no warn-and-continue. Advisory warns; deletes target nothing.

Q-M2-24  **D** — -refresh-only previews and syncs drift explicitly.

 **WHY** — Read-only reconciliation on demand. Destroy/init/format are different modes entirely.

 M2-25 → M2-28

Q-M2-25  **A** — apply -refresh-only writes synced state toward reality.

 WHY — Explicit reconciliation persists. Nothing-written claims deny the sync; backends and trash-plans are unrelated.

Q-M2-26  **B** — Remote operations run off-laptop with history.

 WHY — Agents execute; the workspace records. No-history, local-only, and no-variables all invert the model.

Q-M2-27  **C** — New workspaces start with empty state.

 WHY — Apply fills them. Copied-prod, all-resources, and error claims all misunderstand isolation.

Q-M2-28  **D** — coalesce() returns the first non-null, non-empty value.

 WHY — Fallback chains in order. Always-last, lists, and nothing misread the function.

✓ M2-29 → M2-32

Q-M2-29 ✓ **A** — `fmt -recursive` covers subfolders too.

💡 WHY — Whole-tree style in one run. Single-file, state, and registry are wrong scopes.

Q-M2-30 ✓ **B** — Module outputs flow upward to root and consumers.

💡 WHY — Declaration crosses the boundary at use. Downward-only, nowhere, and S3-direct all invert the contract.

Q-M2-31 ✓ **C** — The root module is the entrypoint config calling children.

💡 WHY — Composition starts here. State files, registries, and lock files are artifacts, not modules.

Q-M2-32 ✓ **D** — `-destroy plan mode` previews teardown for later apply.

💡 WHY — Serialized death list, reviewed first. Creation, init, and formatting are other modes.

✓ M2-33 → M2-36

Q-M2-33 ✓ **A** — auto.tfvars vs tfvars: layered lexical overlays vs single file.

💡 WHY — Multi-file layering in name order. Nothing/comments/speed miss the loading mechanic.

Q-M2-34 ✓ **B** — Piped console tests functions and interpolation with no cloud.

💡 WHY — Stdin evaluation, zero context. Full applies, locks, and billing are heavier verbs.

Q-M2-35 ✓ **C** — Ansible lacks state and plan for infrastructure.

💡 WHY — No binding table, no dry-run diffs. Newer/faster/pricier are non-answers.

Q-M2-36 ✓ **D** — Dynamic blocks repeat nested blocks from variables.

💡 WHY — Data in, stanzas out. Providers, workspaces, and backends repeat at other layers.

✓ M2-37 → M2-40

Q-M2-37 ✓ **A** — TFC cost estimates appear on PRs from remote plans.

💡 WHY — Priced diffs where review happens. State, fmt, and never are wrong venues.

Q-M2-38 ✓ **B** — Prod replaces bare apply with apply tfplan after review.

💡 WHY — Serialized reviewed diff executes. Destroy, rm, and console edits are anti-patterns.

Q-M2-39 ✓ **C** — Soft-mandatory with rights proceeds with permission.

💡 WHY — Override with authority, audited. Always-blocks, deletes, and ignores misread the mode.

Q-M2-40 ✓ **D** — state list shows every managed address.

💡 WHY — The full binding inventory. Bills, sources, and nothing-useful deny its job.

✓ M2-41 → M2-44

Q-M2-41 ✓ **A** — Missing registry pin risks floating majors breaking runs.

💡 WHY — Unpinned floats every release. Nothing/instant/state-loss all misprice the risk.

Q-M2-42 ✓ **B** — Committed tfvars secrets: gitignore + example template + secret store.

💡 WHY — History rewritten, pattern fixed, secrets relocated. Nothing-normal, git-delete, and filename games all fail.

Q-M2-43 ✓ **C** — depends_on belongs on invisible side-effects only.

💡 WHY — References first, meta-argument second. Everywhere-clutter and nowhere-purism both break.

Q-M2-44 ✓ **D** — Cloudflare CNAME in one apply shows multi-provider power.

💡 WHY — Two APIs, one graph, one run. Lock-failure/bug/drift all misread composition.

✓ M2-45 → M2-48

Q-M2-45 ✓ **B** — validate needs no cloud — syntax only.

💡 WHY — Local gate, zero credentials. Cloud/backend/state claims invent requirements.

Q-M2-46 ✓ **B** — Backend blocks take literals or -backend-config only.

💡 WHY — No variables allowed inside backend blocks — partial config plus flags instead. Vars-aren't-broken; S3/speed are unrelated.

Q-M2-47 ✓ **C** — toset() converts lists to stable sets.

💡 WHY — for_each needs set/map identity. Map-to-string, number-to-bool, and nothing misread the cast.

Q-M2-48 ✓ **D** — show reads state or plan files, human or JSON.

💡 WHY — Inspection across formats. DNS, futures, and billing are wrong universes.

 M2-49 → M2-52

Q-M2-49  **A** — TRACE is loudest: full RPC detail.

 WHY — Deepest debugging verbosity. OFF silences; quieter/errors-only invert the scale.

Q-M2-50  **B** — Untracked EC2: state list confirms absence, then import.

 WHY — Prove it's foreign before adopting. Destroy-first, rm-rf, and fmt are panic or nonsense.

Q-M2-51  **C** — TFC run history lives in the workspace.

 WHY — Config plus vars plus state plus runs. Nowhere/local-only/git all deny the platform.

Q-M2-52  **D** — Child outputs consume as module.NAME.output.

 WHY — Like any value once declared. Email, console-only, and never invent ceremony.

 M2-53 → M2-57

Q-M2-53  **D** — Same model, same env = repeatability.
 **WHY** — Identical outcomes from versioned code. Risk/slowness/cost are non-answers.

Q-M2-54  **B** — Workspaces isolate state per env under one backend.
 **WHY** — Same code, N bindings. Providers, binaries, and nothing are un-isolated.

Q-M2-55  **C** — -out then apply gives reviewed == executed.
 **WHY** — Serialized exact diff runs. Faster-init/no-state/no-backend all deny the file.

Q-M2-56  **D** — one() over empty returns null.
 **WHY** — Single-or-null semantics for conditional resources. Errors, lists, and zeros misread it.

Q-M2-57  **A** — Modern refresh = plan -refresh-only.
 **WHY** — Explicit drift preview/sync. Removed/destroy/fmt all misremember history.

✓ M3-01 → M3-04

Q-M3-01 ✓ **A** — Build, change, version = Terraform.

💡 WHY — The three-verb signature. Packers bake, Ansibles configure, gits version code — not infra.

Q-M3-02 ✓ **B** — Private provider registry hosts blessed org providers.

💡 WHY — Internal mirror of approved builds. Public-only, skip-init, and billing miss the point.

Q-M3-03 ✓ **C** — init -upgrade re-resolves newest allowed providers, rewrites lock.

💡 WHY — Controlled freshness within constraints. Destroys/formats/workspaces are other verbs.

Q-M3-04 ✓ **D** — Saved destroy plans via plan -destroy -out.

💡 WHY — Serialized teardown for review. Login/fmt/console are unrelated verbs.

✓ M3-05 → M3-08

Q-M3-05 ✓ **A** — Pinned registry source = best practice.

💡 WHY — Reproducible installs, no floating majors. Optional/forbidden/slow all invert the rule.

Q-M3-06 ✓ **B** — Remote benefits EXCLUDE local-only secrecy.

💡 WHY — EXCEPT-format: shared truth, locking, and off-laptop secrets are real; local secrecy contradicts sharing.

Q-M3-07 ✓ **C** — toset() needed: for_each takes sets or maps only.

💡 WHY — Lists re-index; the cast buys stable identity. Happily/numbers/nothing deny the constraint.

Q-M3-08 ✓ **D** — Ghost-lock recovery starts by reading lock info.

💡 WHY — Who, what, since — before any action. Reboots, table deletes, and destroys are panic order.

✓ M3-09 → M3-12

Q-M3-09 ✓ **A** — Credentials belong in env vars or SSO, never git.
💡 WHY — Environment-scoped secrets. Committed blocks, state outputs, and lock files all leak.

Q-M3-10 ✓ **B** — OS users and packages on live hosts = Ansible.
💡 WHY — Config-management layer, existing fleet. Provisioners, CFN-single-cloud, and TFC venue all miss.

Q-M3-11 ✓ **C** — Conditional idiom: count = var.on ? 1 : 0.
💡 WHY — Ternary-gated existence with safe reads. No-conditionals denies HCL; always-on inverts the ask.

Q-M3-12 ✓ **D** — Infra peer review happens via VCS PRs on HCL.
💡 WHY — Reviewable text, reviewable diffs. Clicks, state edits, and calls bypass review.

✓ M3-13 → M3-16

Q-M3-13 ✓ **A** — `values(map)[*].id` collects all IDs as a list.

💡 WHY — Values projection then splat. Keys, nothing, and one-string misread the two-step.

Q-M3-14 ✓ **B** — Agents run plans remotely with history.

💡 WHY — Off-laptop execution, recorded runs. Laptops, never, and fmt-only invert the model.

Q-M3-15 ✓ **C** — Committed explicit credentials risk scraped secrets in minutes.

💡 WHY — Bots scan public history instantly. Nothing/faster/better-state deny the threat model.

Q-M3-16 ✓ **D** — Stale state reconciles via refresh during plan.

💡 WHY — Reality re-read into the diff. New workspaces, config deletes, and fmt are wrong repairs.

 M3-17 → M3-20

Q-M3-17  **A** — Speculative PR plans come from VCS-driven runs.
 WHY — Remote dry-runs on pull requests. Cron-fmt, local state, and email hooks are unrelated.

Q-M3-18  **B** — state show vpc reads one resource's attributes.
 WHY — Single-address inspection. Bills, sources, and nothing deny its job.

Q-M3-19  **C** — merge() collisions: right side wins.
 WHY — Later arguments override. Left-wins, errors, and neither-drops misremember.

Q-M3-20  **D** — tfvars vs auto.tfvars: single file vs layered lexical overlays.
 WHY — Explicit -var-file vs automatic layering. Nothing/color/speed miss the loading mechanic.

✓ M3-21 → M3-24

Q-M3-21 ✓ **A** — Root passes inputs via module call arguments.
💡 WHY — The only door in. Surgery, consoles, and email bypass the contract.

Q-M3-22 ✓ **B** — Console tag drift appears as ~ change.
💡 WHY — Refresh imports reality; plan proposes revert. No-output/new-backend/crash deny refresh.

Q-M3-23 ✓ **C** — coalesce() picks the first non-null value.
💡 WHY — Ordered fallback chain. Last/all/none misread the function.

Q-M3-24 ✓ **D** — Example tfvars commits; real tfvars stays local.
💡 WHY — Template shared, secrets local. Speed/state/law are non-reasons.

✓ M3-25 → M3-28

Q-M3-25 ✓ **A** — Interrupted apply: review partial state, re-apply.
💡 WHY — Reconciliation through planning. Deletes, new backends, and fmt are panic moves.

Q-M3-26 ✓ **B** — No manual snowflakes = consistency.
💡 WHY — Identical systems from code. Cost/slowness/risk are non-answers.

Q-M3-27 ✓ **C** — Go binary, no runtime = the Terraform CLI distribution.
💡 WHY — Single static executable. Provider-only, state-file, and Sentinel readings miss the subject.

Q-M3-28 ✓ **D** — Two green 0/0/0 applies = idempotence.
💡 WHY — Repeat no-op from unchanged code. Import/drift/taint are other concepts.

✓ M3-29 → M3-32

Q-M3-29 ✓ **A** — try() guards optional access without crashing.

💡 WHY — Fallback on missing paths. Billing, locks, and DNS are wrong universes.

Q-M3-30 ✓ **B** — Graph edges derive from references.

💡 WHY — Interpolation orders creation. File order, comments, and filenames order nothing.

Q-M3-31 ✓ **C** — console tests expressions with no cloud.

💡 WHY — REPL evaluation, zero context. Locks, deploys, and bills are heavier verbs.

Q-M3-32 ✓ **D** — TFC workspaces bind config, vars, state, runs.

💡 WHY — Full execution context. Binaries, suffix-only, and nothing deny the platform.

 M3-33 → M3-36

Q-M3-33  **A** — Multi-cloud in one apply: providers are plugins over RPC.

 **WHY** — One core, many translators, one graph. Giant-binary/no-state/YAML all deny the architecture.

Q-M3-34  **B** — PR cost control via remote-run estimates.

 **WHY** — Priced diffs at review time. Fmt, rm, and taint price nothing.

Q-M3-35  **C** — Splat over instances gives a list of the attribute.

 **WHY** — Projection across the fleet. Maps, errors, and backends are wrong shapes.

Q-M3-36  **D** — Ghost lock: first read lock info — who, what, since.

 **WHY** — Diagnose before acting. Fmt, nothing, and billing skip the evidence.

 M3-37 → M3-40

Q-M3-37  **A** — Deep debugging verbosity = TRACE.
 WHY — Full RPC detail. ERROR/WARN filter; OFF silences.

Q-M3-38  **B** — apply -refresh-only syncs state toward reality.
 WHY — Explicit reconciliation persists. Style, providers, and regions are other verbs.

Q-M3-39  **C** — Advisory breach logs; the run continues.
 WHY — Informational by design. Blocks, deletes, and bills belong to other modes.

Q-M3-40  **D** — Partial apply then re-apply continues from recorded state.
 WHY — No rollback exists; reconciliation moves forward. Rollback/fork/wipe deny the model.

✓ M3-41 → M3-44

Q-M3-41 ✓ **A** — Golden images with software baked = Packer.

💡 WHY — Bakes AMIs; Terraform provisions, provisioners configure last-resort. Console and mv build nothing.

Q-M3-42 ✓ **B** — for_each over a list: wrap with toset().

💡 WHY — Sets give stable identity; lists re-index. Deletes, depends_on, and count-only dodge the cast.

Q-M3-43 ✓ **C** — TF_VAR objects need single-quoted HCL.

💡 WHY — Shell-safe object syntax. Bare words split; binaries and no-quoting are nonsense.

Q-M3-44 ✓ **D** — Child outputs consume as module.NAME.output.

💡 WHY — Like any value once declared. Never/console/email invent ceremony.

 M3-45 → M3-48

Q-M3-45  **A** — terraform.tfvars holds auto-loaded variable values.

 **WHY** — Convention-loaded inputs. Backends, locks, and bills are other files' jobs.

Q-M3-46  **B** — Per-env lock tables buy parallel applies at table cost.

 **WHY** — Isolated serialization per env. Nothing/no-init/free-state misprice the trade.

Q-M3-47  **C** — Validation blocks run at plan.

 **WHY** — Fail-fast input gates. Never/after-destroy/in-git all miss the timing.

Q-M3-48  **D** — -destroy plan mode previews teardown.

 **WHY** — Serialized death list for review. Login/creation/fmt are other modes.

 M3-49 → M3-53

Q-M3-49  **A** — New workspace state starts empty; apply fills it.
 **WHY** — Isolation from zero. Prod-data, errors, and forever-locks deny the model.

Q-M3-50  **B** — TFC workspaces bind config, vars, state, runs.
 **WHY** — Full execution context. Suffix-only, nothing, and binaries describe other things.

Q-M3-51  **C** — Login writes tokens to credentials.tfrc.json.
 **WHY** — Home-dir CLI credentials. State, lock, and tfvars are wrong files.

Q-M3-52  **D** — Locals derive values to avoid repetition — single source.
 **WHY** — Computed once, referenced widely. Duplication, slowness, and state-skips invert the purpose.

Q-M3-53  **A** — terraform output shows exports after apply.
 **WHY** — Values for humans and pipelines. Billing, locks, and nothing deny its job.

 M3-54 → M3-57

Q-M3-54  **B** — Pin registry modules: floating majors break runs.
 WHY — Reproducibility demands pins. Speed, state-needs, and fmt-does are non-reasons.

Q-M3-55  **C** — Soft-mandatory with rights proceeds when permitted.
 WHY — Override with authority, audited trail. Blocks/deletes/ignores misread the mode.

Q-M3-56  **D** — -auto-approve belongs in gated pipelines only.
 WHY — Reviewed plan plus automation gate. Laptops, never-anywhere, and always mis-scope it.

Q-M3-57  **A** — S3 backend state with secrets is plaintext — restrict plus Vault/SSM.
 WHY — No at-rest encryption by Terraform. Encrypted-absent-hashed are all fictions.

Know Your Number: 40

57 questions · 70% to pass · **40 correct minimum**. Score every attempt below. Trend matters more than any single run: three straight 44+ means book the exam.

Attempt	Mock 1 /57	Mock 2 /57	Mock 3 /57	Weakest domain	Drill page to revisit
1st pass	__/57	__/57	__/57	_____	_____
2nd pass	__/57	__/57	__/57	_____	_____
Final	__/57	__/57	__/57	_____	_____

◆ Domain drill log (90 drills · target 80%+ per domain)

Domain	Drills	Score 1	Score 2	Ready?
D1 IaC concepts	6	__/6	__/6	☐
D2 Purpose	8	__/8	__/8	☐
D3 Basics	9	__/9	__/9	☐
D4 CLI	8	__/8	__/8	☐
D5 Modules	8	__/8	__/8	☐
D6 Workflow	9	__/9	__/9	☐
D7 State	18	__/18	__/18	☐
D8 Config	16	__/16	__/16	☐
D9 Cloud	8	__/8	__/8	☐

✓ **READY RULE**

Every domain 80%+ AND two mocks at 44+ on different days. One weak domain fails exams — the blueprint weights breadth.

The Night Before & The 60 Minutes

You carried state, tamed locks, priced plans, and survived 261 questions. The exam is just one more run — with a timer.

🌙 Night before

- ① Re-read the **time math** — 63s/question, three passes.
- ② Skim every **drill key** WHY line once.
- ③ Sleep. Cramming locks costs more than it buys.

🕒 The 60 minutes

- ① Pass 1: answer everything, flag the unsure.
- ② Pass 2: return to flags with fresh eyes.
- ③ Pass 3: review EXCEPT/NOT stems — they flip meanings.
- ④ Never leave blanks — no penalty for wrong.

🔍 TRAP RADAR

EXCEPT/NOT stems · workspace homonyms (CLI vs TFC) · -var precedence tops · sensitive-masks-display vs encryption · mandatory-blocks vs advisory-logs.

📚 SERIES RESOURCES

Vol 1 Foundations · Vol 2 Production · Practice Lab · this Command Center — the full Terraform Master path by Eng. Mohamed Magdy's course.

— The End 📺 — Good luck. Go take the 40.